

Gabriel Silveira de Azevedo
Mateus Carvalho Costa Nogueira

**Desenvolvimento de uma API RESTful para
captação e recomendação de potenciais
doadores de sangue utilizando Domain Driven
Design (DDD)**

Campos dos Goytacazes

2026

Gabriel Silveira de Azevedo
Mateus Carvalho Costa Nogueira

**Desenvolvimento de uma API RESTful para captação e
recomendação de potenciais doadores de sangue
utilizando Domain Driven Design (DDD)**

Monografia apresentada ao Instituto Federal de
Educação, Ciência e Tecnologia Fluminense
como parte das exigências para a conclusão do
Curso de Bacharelado em Sistemas de Informa-
ção.

Instituto Federal de Educação, Ciência e Tecnologia Fluminense
Campus Campos Centro
Curso de Bacharelado em Sistemas de Informação

Orientador: Mark Douglas Azevedo Jacyntho

Campos dos Goytacazes
2026

Gabriel Silveira de Azevedo
Mateus Carvalho Costa Nogueira

Desenvolvimento de uma API RESTful para captação e recomendação de potenciais doadores de sangue utilizando Domain Driven Design (DDD)

Monografia apresentada ao Instituto Federal de
Educação, Ciência e Tecnologia Fluminense
como parte das exigências para a conclusão do
Curso de Bacharelado em Sistemas de Informa-
ção.

Trabalho aprovado. Campos dos Goytacazes, _____ de _____ de
2026.

Mark Douglas Azevedo Jacyntho
Orientador

Membro Titular 1
Avaliador

Membro Titular 2
Avaliador

Lista de abreviaturas e siglas

API	Application Programming Interface
DDD	Domain Driven Design
FIFO	First In, First Out
JWT	JSON Web Token
LTS	Long Term Support
NoSQL	Not Only SQL
OMS	Organização Mundial da Saúde
OO	Orientação a Objetos
OPAS	Organização Pan-Americana da Saúde
REST	Representational State Transfer
SOLID	Single responsibility, Open-closed, Liskov substitution, Interface segregation, Dependency inversion
TIC	Tecnologias da Informação e Comunicação
UML	Unified Modeling Language

Lista de ilustrações

Figura 1 – Distribuição de artigos por ano	22
Figura 2 – Distribuição de artigos por país	22
Figura 3 – Diagrama de processos do desenvolvimento do TCC	27
Figura 4 – Diagrama de processos do Desenvolvimento da API	28
Figura 5 – Diagrama de caso de uso de visitante	36
Figura 6 – Diagrama de caso de uso de requisitante	37
Figura 7 – Diagrama de caso de uso de doador	38
Figura 8 – Diagrama conceitual dos participantes	39
Figura 9 – Diagrama conceitual do processo de doação	40
Figura 10 – Diagrama de implementação: agregado DonationRequest	41
Figura 11 – Diagrama de implementação: agregado Donation	42

Lista de tabelas

Tabela 1 – Classificação das palavras-chave	20
Tabela 2 – Expressão de busca usada no Scopus	21
Tabela 3 – Critérios de inclusão e exclusão	21
Tabela 4 – Lista de artigos selecionados	23
Tabela 5 – Cronograma de execução da primeira etapa do TCC	31
Tabela 6 – Cronograma de execução da segunda etapa do TCC	31
Tabela 7 – Requisitos funcionais	33
Tabela 8 – Requisitos não funcionais	34
Tabela 9 – Lista de casos de uso	35
Tabela 10 – Entidades e suas responsabilidades	43

Resumo

A transfusão sanguínea é essencial em emergências, cirurgias e tratamentos crônicos, mas o Brasil ainda enfrenta descompasso entre demanda e oferta de doadores. Este trabalho propõe o desenvolvimento de uma API RESTful para captação de doadores e recomendação de solicitações compatíveis, utilizando DDD e princípios SOLID. A pesquisa adota abordagem qualitativa, exploratória-descritiva, com mapeamento sistemático da literatura e modelagem do domínio da doação sanguínea. A solução, implementada em Java 17 com Spring Boot, MongoDB e autenticação JWT, intermedia requisitantes e doadores, considerando tipagem sanguínea, proximidade geográfica e alocação FIFO de doações às metas das solicitações.

Palavras-chave: doação de sangue; API RESTful; Domain-Driven Design; recomendação de solicitações; engenharia de software.

Abstract

Blood transfusion is essential in emergencies, surgeries and chronic treatments, yet Brazil still faces a mismatch between demand and donor supply. This undergraduate thesis proposes a RESTful API for donor recruitment and recommendation of compatible donation requests, using DDD and SOLID principles. The research adopts a qualitative, exploratory-descriptive approach, with a systematic literature mapping and blood-donation domain modeling. The solution, implemented in Java 17 with Spring Boot, MongoDB and JWT authentication, mediates between requesters and donors, considering blood type, geographic proximity and FIFO allocation of donations to request goals.

Keywords: blood donation; RESTful API; Domain-Driven Design; request recommendation; software engineering.

Sumário

1	INTRODUÇÃO	10
1.1	Problema e contexto	10
1.2	Objetivos	10
1.3	Justificativa	11
1.4	Organização da monografia	11
2	REFERENCIAL TEÓRICO	12
2.1	Orientação a Objetos	12
2.1.1	Abstração	12
2.1.2	Encapsulamento	13
2.1.3	Herança	13
2.1.4	Polimorfismo	14
2.2	Princípios SOLID	14
2.2.1	<i>Single Responsibility Principle</i> (SRP)	15
2.2.2	<i>Open/Closed Principle</i> (OCP)	15
2.2.3	<i>Liskov Substitution Principle</i> (LSP)	15
2.2.4	<i>Interface Segregation Principle</i> (ISP)	16
2.2.5	<i>Dependency Inversion Principle</i> (DIP)	16
2.3	<i>Domain-Driven Design</i> (DDD)	16
2.3.1	Padrões de <i>design</i> estratégico	17
2.3.2	Padrões de <i>design</i> tático	17
2.3.3	Arquitetura e isolamento do domínio	18
3	TRABALHOS RELACIONADOS	19
3.1	Protocolo de pesquisa	19
3.1.1	Objetivo e questões de pesquisa	19
3.1.2	Instrumentos	20
3.1.3	Critérios de seleção e procedimentos	20
3.2	Caracterização da pesquisa	21
3.3	Resultados e discussões	23
3.4	Considerações	25
4	MÉTODOS E RECURSOS	26
4.1	Visão geral	26
4.2	Instrumentos e ferramentas	29
4.2.1	Pesquisa exploratória-descritiva	29

4.2.2	Desenvolvimento da API	30
4.2.2.1	Linguagem de programação (Java)	30
4.2.2.2	<i>Framework</i> de desenvolvimento (Spring Boot)	30
4.2.2.3	Ambiente de desenvolvimento e versionamento (Visual Studio Code e Git)	30
4.3	Cronograma	31
5	ESTUDO DE CASO: REQUISITOS, MODELAGEM E IMPLEMENTAÇÃO	32
5.1	Visão geral da solução	32
5.2	Levantamento de requisitos	32
5.2.1	Requisitos funcionais	32
5.2.2	Requisitos não funcionais	33
5.3	Modelagem do sistema	34
5.3.1	Casos de uso	34
5.3.2	Diagramas de classes	38
5.4	Domínio e arquitetura	42
5.4.1	Linguagem ubíqua e organização em camadas	42
5.4.2	Entidades, agregados e objetos de valor	43
5.5	Implementação	44
5.5.1	Persistência de dados	44
5.5.2	Segurança e autenticação	44
5.5.3	Serviços externos e documentação da API	45
5.6	Recomendação de solicitações e cumprimento de metas	45
5.6.1	Recomendação de solicitações para o doador	45
5.6.2	Notificação de doadores potenciais	46
5.6.3	Materialização do cumprimento de metas (fulfilledBloodBags)	46
6	AVALIAÇÃO DA PROPOSTA DE SOLUÇÃO	48
6.1	Plano de avaliação	48
6.2	Resultados	48
6.3	Sugestões de melhorias	48
6.4	Considerações sobre a avaliação	49
7	CONSIDERAÇÕES FINAIS	50
7.1	Alcance dos objetivos	50
7.2	Principais contribuições	50
7.3	Trabalhos futuros	50
	REFERÊNCIAS	52

1 Introdução

1.1 Problema e contexto

A transfusão sanguínea é essencial em emergências, cirurgias de grande porte e tratamentos de condições crônicas. Mantê-la depende de doações periódicas e de mecanismos eficazes de captação.

Segundo a Organização Mundial da Saúde (OMS) e a Organização Pan-Americana da Saúde (OPAS) Organização Mundial da Saúde (2020), Organização Pan-Americana da Saúde (2020), a taxa de doação por mil habitantes varia de 32,6 em países de alta renda a 4,4 em países de baixa renda. No Brasil, a demanda permanece elevada frente à oferta de doadores.

Do ponto de vista tecnológico, a conexão entre oferta e demanda ainda é limitada. Solicitações emergenciais em redes sociais evidenciam a dificuldade de localizar doadores adequados com rapidez e a baixa conversão dessas chamadas em doações efetivas. É necessário, portanto, desenvolver soluções escaláveis e bem estruturadas, com ciclo de vida longo.

1.2 Objetivos

O objetivo geral deste trabalho é desenvolver uma *Application Programming Interface* (API) RESTful (*Representational State Transfer*) para apoiar a captação de doadores a partir de solicitações de unidades de saúde e pacientes. A API atua como intermediadora entre demanda (requisitantes) e oferta (doadores), com recomendação de solicitações compatíveis — considerando tipagem sanguínea e proximidade geográfica — e identificação de doadores elegíveis para notificação.

Objetivos específicos:

- O1** Mapear o processo da doação sanguínea com técnicas de *Domain-Driven Design* (DDD).
- O2** Identificar sistemas com funcionalidades similares.
- O3** Desenvolver mecanismos de recomendação de solicitações a doadores e de notificação a candidatos elegíveis.
- O4** Validar a proposta por meio de estudo de caso e implementação da API.
- O5** Avaliar a manutenibilidade da solução.

1.3 Justificativa

A OMS recomenda que ao menos 1% da população doe sangue Organização Mundial da Saúde (2020). Mesmo quando índices nominais são atingidos, a oferta pode ser insuficiente Hemorio (2023). O Ceará, com 2,21% da população doadora Ceará. Secretaria da Saúde (2021), ilustra o efeito de uma hemorrede integrada coordenada pelo Hemoce, com mais de 480 unidades de saúde Centro de Hematologia e Hemoterapia do Ceará — HEMOCE (2022) e recorde de 110.784 doações em 2024 Ceará. Secretaria da Saúde (2025).

No plano técnico, o DDD favorece a modelagem fiel das regras do domínio Evans (2003), e os princípios SOLID Martin (2000) apoiam evolução e integração. A contribuição deste trabalho é uma API RESTful orientada a esse domínio, voltada à captação efetiva de doadores.

1.4 Organização da monografia

A Seção 2 apresenta o referencial teórico (orientação a objetos, SOLID e DDD). A Seção 3 relata o mapeamento sistemático da literatura e os trabalhos relacionados. A Seção 4 descreve métodos, instrumentos e cronograma. A Seção 5 detalha requisitos, modelagem e implementação. A Seção 6 trata da avaliação. A Seção 7 reúne as considerações finais.

2 Referencial teórico

Este capítulo apresenta os pilares teóricos que alicerçam a arquitetura e o desenvolvimento do sistema de API proposto, sendo dividido em três tópicos complementares: primeiramente, os princípios da Orientação a Objetos (OO), que trazem a base conceitual sobre a qual todo o desenvolvimento é edificado; em seguida, são abordados os princípios SOLID, que fortalecem as diretrizes para o *design* das classes e módulos, tornando-os mais flexíveis e manuteníveis; e, por fim, é apresentado o *Domain-Driven Design* (DDD), que orienta o desenvolvimento e o entendimento do sistema de forma alinhada ao domínio do negócio. A decisão por apresentá-los nessa ordem não é aleatória: cada tema é tratado como uma camada que complementa a anterior, formando um conjunto de conhecimentos coeso e aplicável a domínios com regras de negócio complexas e relativamente estáveis, como é o caso da doação sanguínea.

2.1 Orientação a Objetos

A OO é um paradigma de programação que tem como objetivo organizar o código em torno de objetos, estruturas que encapsulam dados e comportamentos, simulando entidades e eventos do mundo real Sommerville (2011). Embora tenha surgido nos anos 1960 com a linguagem de programação Simula, a OO só ganhou força mais tarde, com linguagens como Smalltalk, C++ e Java, estabelecendo-se como paradigma fundamental no desenvolvimento de *software* Booch (2007). A disseminação desse paradigma na comunidade de desenvolvedores decorre de sua capacidade de modelar problemas do mundo real de forma intuitiva, aproximando a estrutura do código e a organização das classes da estrutura do problema que se pretende resolver Sommerville (2011).

A OO é estruturada em quatro pilares fundamentais e complementares: abstração, encapsulamento, herança e polimorfismo.

2.1.1 Abstração

A abstração permite que os programadores se preocupem apenas com as características essenciais de um objeto, ignorando detalhes desnecessários ao modelo e ao contexto em questão Booch (2007). Em vez de lidar com cada detalhe, desenvolvedores trabalham em níveis mais altos de abstração. Isso ocorre porque, segundo Sommerville (2011), a abstração ajuda a controlar a complexidade do *software*, mantendo apenas o essencial do domínio na visão dos programadores.

A definição de contratos entre tipos surge principalmente por meio de classes abstratas e interfaces, que estabelecem obrigações funcionais sem detalhar o processo interno Gamma et

al. (1994). Implementações concretas surgem depois, moldadas conforme as particularidades de cada classe descendente.

No domínio da doação sanguínea, a abstração pode ser exemplificada pela definição de um conceito genérico de participante do processo, que reúna características comuns a diferentes atores — como nome, contato e endereço — independentemente de se tratar de um doador, de um solicitante ou de uma instituição de saúde. Especializações desse conceito genérico assumiriam, então, comportamentos e regras próprias de cada papel, sem que o restante do sistema precisasse conhecer tais particularidades para lidar com o participante em um nível mais alto de abstração.

2.1.2 Encapsulamento

O encapsulamento é responsável por controlar o acesso aos dados internos de um objeto, exigindo que qualquer interação aconteça por meio de vias específicas, o que preserva a coerência do estado interno Sommerville (2011). Ainda segundo Booch (2007), esconder os detalhes de um objeto que não contribuem para suas características essenciais faz parte do conceito, de maneira que a interface pública e a implementação interna permaneçam separadas, permitindo alterações na implementação que não afetam o código cliente que utiliza a classe Sommerville (2011).

De maneira prática, o encapsulamento usa modificadores de acesso para definir quais atributos e métodos podem ser vistos ou utilizados externamente. Assim, os dados ficam protegidos, enquanto métodos específicos permitem interação controlada. De acordo com Bloch (2018), essa prática permite validações e garante a consistência do objeto.

No domínio da doação sanguínea, o encapsulamento se manifestaria, por exemplo, na proteção de regras associadas à elegibilidade de um doador ou à validade de uma solicitação de sangue: informações como o tipo sanguíneo, a data da solicitação e o prazo limite para atendimento permaneceriam protegidas de alterações diretas, sendo modificadas apenas por meio de operações que garantam a consistência do estado do objeto, como a validação de que o prazo limite não seja anterior à data da solicitação.

2.1.3 Herança

O conceito de herança possibilita o reaproveitamento e a especialização de comportamentos por subclasses, sendo fundamental para a reutilização de código e o estabelecimento de relações hierárquicas entre tipos Sommerville (2011). Segundo Booch (2007), a herança funciona como o meio pelo qual uma classe herda métodos e propriedades de outra, concretizando vínculos do tipo “é-um” entre entidades. Dessa forma, surgem arranjos hierárquicos que organizam classes representando relacionamentos ontológicos do contexto do problema.

A herança pode ocorrer de duas principais maneiras: a herança simples, em que uma classe deriva de uma única superclasse, e a herança múltipla, em que uma classe herda de mais

de uma superclasse, recebendo todos os atributos e métodos não privados das classes-base. Em ambos os casos é possível sobrescrever métodos para implementação específica Booch (2007). Sommerville (2011) destaca que a herança facilita a manutenção do sistema, uma vez que mudanças na superclasse são automaticamente propagadas para as subclasses.

No domínio da doação sanguínea, a herança poderia ser aplicada ao se derivar diferentes tipos de participantes — como pessoas físicas e instituições — de um conceito comum que concentre atributos compartilhados, permitindo que cada tipo especializado adicione apenas as características próprias de seu contexto, sem duplicar o que já é comum a todos.

2.1.4 Polimorfismo

Formas distintas de um mesmo comportamento surgem quando objetos diversos utilizam uma interface idêntica, conforme descrito por Booch (2007). Segundo Sommerville (2011), o polimorfismo permite que métodos funcionem de formas diversas em diferentes níveis de abstração, característica essencial na programação orientada a objetos.

Existem dois tipos principais de polimorfismo: o polimorfismo de sobrecarga (*overloading*), em que operações distintas com o mesmo nome coexistem com assinaturas diferentes, e o polimorfismo de sobrescrita (*overriding*), em que subclasses redefinem métodos herdados da superclasse Booch (2007). O polimorfismo de subtipo, também conhecido como polimorfismo de inclusão, permite que uma referência de uma superclasse ou interface aponte para objetos de qualquer uma de suas subclasses, possibilitando que o sistema determine em tempo de execução qual implementação específica deve ser invocada Sommerville (2011). Essa característica reduz o acoplamento entre componentes e facilita a adição de novos tipos ao sistema sem modificar código existente. Para que o polimorfismo de subtipo funcione corretamente, é fundamental que as classes derivadas possam substituir suas classes mãe sem alterar o comportamento esperado do sistema, princípio conhecido como *Liskov Substitution Principle* (LSP), segundo o qual objetos de uma classe derivada devem poder ser usados no lugar de objetos da classe base sem comprometer a correção do programa Martin (2003).

No domínio da doação sanguínea, o polimorfismo se manifestaria ao permitir que diferentes tipos de participantes — pessoas físicas e instituições, por exemplo — fossem tratados de maneira uniforme por meio de uma interface comum, possibilitando que o sistema execute operações genéricas, como o envio de notificações, sem a necessidade de verificações explícitas sobre o tipo concreto de cada participante.

2.2 Princípios SOLID

Com origem na década de 2000, SOLID — acrônimo para *Single Responsibility*, *Open/Closed*, *Liskov Substitution*, *Interface Segregation* e *Dependency Inversion* — reúne cinco princípios fundamentais voltados ao *design* orientado a objetos em desenvolvimento de *software*, origi-

nalmente pensados por Robert C. Martin (2003). Posteriormente, em 2004, Michael Feathers organizou essas contribuições na abreviação que tornou o conjunto mais acessível pelo acrônimo SOLID.

Partindo dos conceitos definidos por Martin, surgem diretrizes capazes de tornar os sistemas mais estruturados, desacoplados conforme a necessidade e mais simples de manter (Martin (2008)). Embora nem sempre aplicado integralmente, segundo Martin (2017), o uso rigoroso dessas diretrizes faz com que o *software* cresça com maior controle, reduzindo consequências indesejadas ao modificar funcionalidades ou responder a novas demandas.

A fundação das diretrizes SOLID está ligada à maneira como elas lidam com falhas típicas no processo de criação de *softwares*, como código rígido, frágil ou pouco reaproveitável (Martin (2003)). Embora cada princípio trate de um aspecto distinto do projeto orientado a objetos, todos convergem para objetivos comuns: diminuir dependências entre partes distintas, fortalecer a coesão interna das classes e permitir expansões futuras sem alterar o que já está pronto. Em domínios com lógica de negócio complexa, como o da doação sanguínea, seguir tais princípios torna-se especialmente relevante.

Abaixo, os cinco princípios supracitados são detalhados um a um.

2.2.1 *Single Responsibility Principle* (SRP)

Este princípio estabelece que uma classe deve possuir apenas uma única razão para mudar, concentrando-se em uma responsabilidade coesa (Martin (2003)). Como destaca Booch (2007), a separação de responsabilidades é essencial para gerir a “complexidade organizada”, garantindo que alterações em uma regra de negócio não propaguem erros inesperados em partes não relacionadas do sistema.

2.2.2 *Open/Closed Principle* (OCP)

Preconiza que as entidades de *software* (classes, módulos, funções) devem estar abertas para extensão, mas fechadas para modificação (Martin (2003)). O objetivo é permitir que o comportamento de um sistema seja estendido sem a necessidade de alterar o código-fonte original, o que é alcançado por meio do uso de abstrações e polimorfismo. Padrões de projeto como *Strategy* e *Decorator* são exemplificações práticas deste princípio (Gamma et al. (1994)).

2.2.3 *Liskov Substitution Principle* (LSP)

Formulado por Barbara Liskov, este princípio define que objetos de uma classe derivada devem ser capazes de substituir objetos da classe base sem comprometer a semântica do programa (Martin (2003)). A conformidade com o LSP garante que a hierarquia de herança seja logicamente consistente, assegurando que o polimorfismo de subtipo não resulte em comportamentos inesperados.

2.2.4 *Interface Segregation Principle (ISP)*

Orienta que os clientes não devem ser forçados a depender de interfaces que não utilizam Martin (2003). Em termos práticos, é preferível criar interfaces específicas e granulares em vez de uma interface única e sobrecarregada. Essa prática minimiza o impacto de mudanças e respeita o conceito de ocultação de informação e encapsulamento Booch (2007).

2.2.5 *Dependency Inversion Principle (DIP)*

Define que módulos de alto nível não devem depender de módulos de baixo nível, mas ambos devem depender de abstrações Martin (2003). O fundamento deste princípio é “programar para uma interface, não para uma implementação”, reduzindo o acoplamento e facilitando a injeção de dependências, o que aumenta a flexibilidade e a testabilidade do sistema Martin (2008).

2.3 *Domain-Driven Design (DDD)*

Proposto originalmente por Evans (2003), o DDD — também conhecido como Projeto Orientado ao Domínio — é uma abordagem de desenvolvimento de *software* voltada para sistemas complexos cuja principal dificuldade reside na lógica de negócio. Por isso, tem como foco central o entendimento profundo das regras de negócio, estabelecendo que o *design* do *software* deve ser guiado por um modelo que reflita fielmente o domínio — o problema central que a aplicação visa resolver.

Segundo Vernon (2013), o DDD não deve ser encarado como uma tecnologia ou metodologia rígida, mas sim como um conjunto de ferramentas que permite lidar com a complexidade no coração do *software*, priorizando o que é estrategicamente mais importante para a organização.

A base fundamental do DDD está na colaboração estreita entre desenvolvedores e especialistas do domínio — aqueles que conhecem todas as nuances do negócio. Para que esse diálogo seja possível e produtivo, o DDD propõe a criação de uma Linguagem Ubíqua: um vocabulário comum e rigoroso, compartilhado por todos os membros da equipe, possibilitando um contato produtivo entre desenvolvedores, que a princípio não conhecem o domínio, e especialistas, que são a fonte de conhecimento para o desenvolvimento. Conforme Vernon (2016), essa linguagem não deve existir apenas em documentos ou diagramas, mas precisa ser falada nas reuniões do dia a dia e refletida diretamente no código-fonte, de modo que a intenção do negócio nunca se perca nas decisões técnicas. No domínio da doação sanguínea, essa linguagem se traduziria na adoção de termos como “doador”, “solicitação de doação” e “tipo sanguíneo” diretamente no vocabulário técnico da equipe, nomes que qualquer especialista da área reconheceria imediatamente, sem necessidade de tradução.

Para organizar esse conhecimento e tornar a abordagem mais acessível na prática, autores como Vernon e Khan estruturam o DDD em três pilares fundamentais — e é justamente essa

divisão que orienta as seções seguintes.

2.3.1 Padrões de *design* estratégico

No DDD, o *design* estratégico fornece diretrizes para a organização do sistema em larga escala, permitindo a identificação de fronteiras claras e o alinhamento do *software* com os objetivos de negócio. O conceito central nesta fase é o Contexto Delimitado (*Bounded Context*), que atua como uma fronteira semântica dentro da qual um modelo específico é aplicável e consistente Evans (2003). A definição dessas fronteiras é essencial para evitar que o sistema degenera em uma estrutura técnica confusa e fortemente acoplada, frequentemente descrita na literatura como uma “Grande Bola de Lama” (*Big Ball of Mud*) Vernon (2013).

Para gerenciar a comunicação entre diferentes contextos, utiliza-se o Mapeamento de Contextos (*Context Mapping*), que define as relações técnicas e organizacionais entre as partes do sistema Vernon (2016). Dentre os padrões de mapeamento, destaca-se a Camada Anticorrupção (*Anticorruption Layer*), que funciona como um mecanismo defensivo para isolar o modelo de domínio principal de influências externas ou sistemas legados, preservando a integridade das regras internas Evans (2003).

No domínio da doação sanguínea, é comum delimitar ao menos dois contextos com preocupações distintas: um voltado à solicitação e à priorização de doações, centrado nas regras de compatibilidade sanguínea e de urgência, e outro voltado à gestão dos doadores, responsável pelas regras de elegibilidade, histórico de doações e comunicação com os candidatos. Uma separação dessa natureza favorece que alterações nas regras de um dos contextos — por exemplo, os critérios de elegibilidade de um doador — não impactem diretamente a lógica do outro.

2.3.2 Padrões de *design* tático

Enquanto o *design* estratégico lida com as fronteiras de larga escala, os padrões de *design* tático fornecem as ferramentas para a modelagem detalhada dentro de um Contexto Delimitado, garantindo que os objetos do sistema possuam comportamento rico e mantenham a consistência dos dados Evans (2003).

O primeiro bloco de construção são as Entidades: objetos definidos por uma identidade única que persiste ao longo do tempo, independentemente de mudanças em seus atributos Vernon (2013). No domínio da doação sanguínea, um solicitante ou um centro de coleta são exemplos de entidades — um solicitante continua sendo o mesmo solicitante mesmo que seu contato ou endereço seja atualizado, pois o que o identifica é sua identidade no domínio, não seus dados.

Em contrapartida, os Objetos de Valor (*Value Objects*) são imutáveis e definidos exclusivamente por seus atributos, sem identidade própria Evans (2003). Conceitos como tipo sanguíneo, endereço ou contato se enquadrariam nessa categoria: dois doadores com o mesmo tipo sanguíneo compartilham o mesmo valor, e não faz sentido distingui-los por identidade —

o que importa é o valor em si. Essa imutabilidade traz segurança ao modelo, pois garante que tais conceitos não sejam alterados de forma inadvertida.

Os Agregados representam agrupamentos de entidades e objetos de valor tratados como uma unidade para fins de consistência e mudança de dados. Cada agregado possui uma Raiz do Agregado (*Aggregate Root*), responsável por garantir que todas as regras de negócio e invariantes sejam satisfeitas antes de qualquer transação ser concluída Vernon (2013). No domínio da doação sanguínea, uma solicitação de doação — com seu ciclo de vida completo, desde a criação até o atendimento — ou o histórico de doações de um doador são exemplos plausíveis de agregados: nenhum elemento interno poderia ser manipulado diretamente por objetos externos, devendo toda interação passar obrigatoriamente pela raiz do agregado, o que preserva a integridade do modelo.

Por fim, os Eventos de Domínio registram ocorrências significativas para o negócio no passado, permitindo a comunicação desacoplada entre diferentes partes do sistema Vernon (2016). No contexto da doação sanguínea, a confirmação de uma doação ou a manifestação de interesse de um candidato são exemplos de eventos capazes de disparar notificações ou atualizar o estado de uma solicitação, sem que os componentes envolvidos precisem se conhecer diretamente.

2.3.3 Arquitetura e isolamento do domínio

Para que o modelo de domínio permaneça livre de contaminações tecnológicas, o DDD recomenda arquiteturas que isolem a lógica de negócio das preocupações de infraestrutura. Padrões como a Arquitetura em Camadas ou a Arquitetura de Portas e Adaptadores — também conhecida como Arquitetura Hexagonal — garantem que a camada de domínio permaneça pura e independente de detalhes técnicos, como bancos de dados ou interfaces de usuário Evans (2003). Esse isolamento facilita a manutenibilidade, a testabilidade e a evolução contínua do conhecimento sobre o negócio dentro do *software* Vernon (2016).

Em um sistema modelado sob essa perspectiva, esse isolamento se refletiria na separação clara entre as classes de domínio — responsáveis por expressar as regras de negócio da doação sanguínea — e os serviços responsáveis por coordenar operações de infraestrutura, como persistência e envio de notificações. Dessa forma, a camada de domínio permaneceria como uma abstração independente de tecnologia, sobre a qual o restante da aplicação é construído.

3 Trabalhos relacionados

Este capítulo apresenta o estudo de trabalhos relacionados, com o propósito de mapear o estado da arte referente a sistemas de apoio à doação de sangue. A investigação busca compreender como a literatura acadêmica tem abordado a captação de doadores e o uso de algoritmos de recomendação nesse cenário. Ao analisar as soluções tecnológicas já publicadas, torna-se possível identificar lacunas, justificar as escolhas arquiteturais adotadas nesta pesquisa e evidenciar a contribuição da API desenvolvida em comparação ao contexto tecnológico atual.

3.1 Protocolo de pesquisa

Para garantir o rigor científico na busca e seleção dos estudos, adotou-se o método de Mapeamento Sistemático da Literatura. Segundo Petersen et al. (2008), essa abordagem fornece uma estrutura objetiva para descobrir, categorizar e estruturar o conhecimento existente sobre um tópico específico dentro da engenharia de software. Este protocolo define os passos executados durante a pesquisa bibliográfica, assegurando que o processo seja replicável e livre de vieses de seleção. O enquadramento metodológico geral da pesquisa — abordagem qualitativa, caráter exploratório-descritivo e instrumentos — é apresentado no Capítulo 4. As subseções a seguir descrevem as questões norteadoras, as bases de dados consultadas e os parâmetros aplicados para a filtragem do material.

3.1.1 Objetivo e questões de pesquisa

O objetivo deste mapeamento sistemático da literatura é identificar, analisar e classificar as soluções tecnológicas existentes que apoiam a doação de sangue, com ênfase em sistemas que utilizam abordagens de recomendação de possíveis doadores. Desse modo, a principal questão de pesquisa que se pretende responder é:

1. Quais são os sistemas/aplicativos que auxiliam na captação de doadores de sangue?

Além disso, visa responder subquestões a partir dos trabalhos selecionados, tais como:

- 1.1) Há sistemas que utilizam algoritmos de recomendação para sugerir ações ou contatos entre doadores e instituições?
- 1.2) Quais são as principais funcionalidades e estratégias de incentivo implementadas em sistemas computacionais para promover a doação de sangue?

3.1.2 Instrumentos

Para identificar os trabalhos foi utilizado o Scopus. De acordo com Elsevier (2024), Scopus é um banco de dados de resumos e citações de literatura revisada por pares, que relatam resultados da pesquisa mundial em áreas como: ciência, tecnologia, medicina, artes e humanidades. O Scopus abrange mais de 2,4 bilhões de referências citadas desde 1970 Elsevier (2025).

3.1.3 Critérios de seleção e procedimentos

Foram realizadas buscas por referenciais bibliográficos, utilizando indexação de dados em bases científicas como CAPES Periódicos (<<http://www.periodicos.capes.gov.br/>>) e Scopus, com descritores como “doação”; “sangue”; “sistema”; “aplicação”; “captação” e “recomendação”. Os critérios de inclusão consideraram publicações entre os anos de 2015 e 2025, em português e inglês, limitando a área temática a medicina, engenharia e ciência da computação, com foco em artigo, revisão e artigos de conferência que apresentassem fundamentação teórica sólida e aplicabilidade prática. A pesquisa foi realizada por assunto, utilizando termos de busca associados a operadores booleanos, a fim de obter resultados na base de informação do Scopus.

A expressão de busca utilizada no Scopus foi definida utilizando a técnica PICO (*Population, Intervention, Comparison, Outcome*). Frandsen (2020) recomenda o uso de PICO para obter uma resposta precisa para a estruturação de buscas de revisões sistemáticas da literatura que incluem as facetas: população, intervenção, comparação e o resultado. O Quadro 1 apresenta a distribuição das palavras-chave da questão de pesquisa nas facetas PICO.

Tabela 1 – Classificação das palavras-chave

Questão	P	I	C	O	C
a.1) Quais são os sistemas/aplicativos que auxiliam na captação de doadores de sangue?	doador* OR donor*	“sangue” OR “blood”	—	“sistema” OR “system” OR “app” OR “application”	“captação” OR “recruitment” OR recommend*

Fonte: Elaborado pelos próprios autores (2025).

Já o Quadro 2 representa a *query* (expressão de busca) que foi utilizada para a busca dos trabalhos relacionados na base de dados do Scopus.

Tabela 2 – Expressão de busca usada no Scopus

```
( TITLE-ABS-KEY ( doador* OR donor* ) AND TITLE-ABS-KEY ( sangue OR
blood ) AND TITLE-ABS-KEY ( Sistema OR system OR app OR application
) AND TITLE-ABS-KEY ( Captação OR recruitment OR recomend* ) ) AND
PUBYEAR > 2014 AND PUBYEAR < 2026 AND ( LIMIT-TO ( SUBJAREA , "COMP")
OR LIMIT-TO ( SUBJAREA , "ENGI") OR LIMIT-TO ( SUBJAREA , "MEDI")
) AND ( LIMIT-TO ( LANGUAGE , "English") OR LIMIT-TO ( LANGUAGE ,
"Portuguese") ) AND ( LIMIT-TO ( DOCTYPE , "ar") OR LIMIT-TO ( DOCTYPE
, "re") OR LIMIT-TO ( DOCTYPE , "cp") )
```

Fonte: Elaborado pelos próprios autores (2025).

Os critérios de inclusão (CI) e exclusão (CE) utilizados para selecionar os artigos são apresentados no Quadro 3.

Tabela 3 – Critérios de inclusão e exclusão

ID	Critério
CI1	Texto completo disponível em PDF ou HTML via acesso institucional.
CI2	O artigo contém informação relevante para responder às questões postuladas.
CE1	Artigo duplicado.
CE2	O artigo não contém informação relevante para responder às questões postuladas.

Fonte: Elaborado pelos próprios autores (2025).

Na primeira etapa foi verificada a disponibilidade de texto completo, a partir do download via Scopus. Depois, a seleção dos artigos com texto completo envolveu a interpretação do título e resumo para verificar se atende ao critério CI2. Simultaneamente, aplicaram-se os critérios de exclusão (CE1 e CE2) de forma rigorosa, descartando as publicações que não se alinhavam ao escopo tecnológico da pesquisa ou que apresentavam duplicidade na base de dados.

3.2 Caracterização da pesquisa

A pesquisa foi realizada em julho de 2025 e retornou 147 documentos. O Gráfico 1 apresenta a distribuição de artigos por ano. Observa-se que a maioria dos artigos encontrados são dos anos de 2020 e 2021.

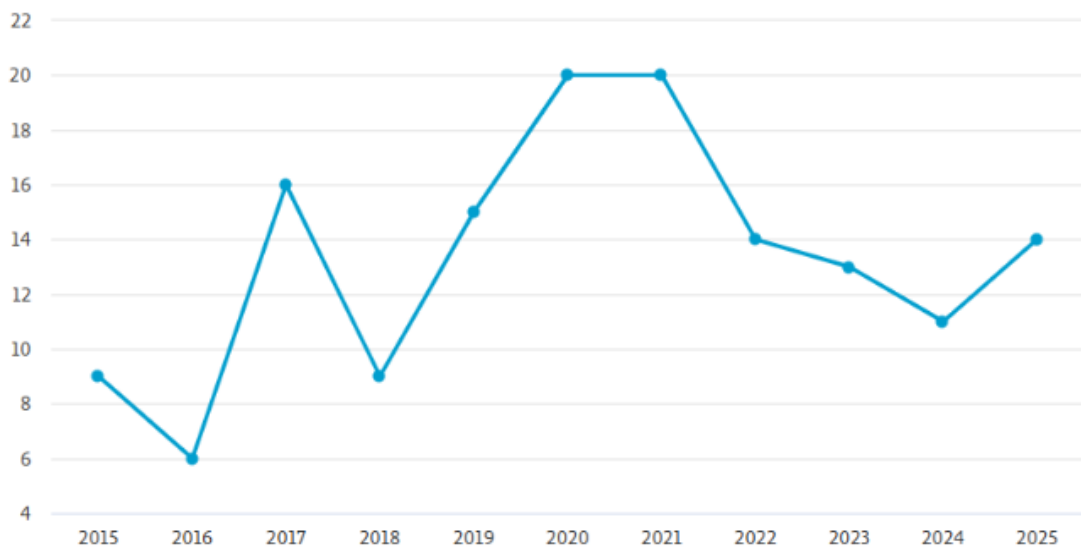


Figura 1 – Distribuição de artigos por ano

Fonte: Scopus (2025).

O Gráfico 2 apresenta a distribuição de artigos por país, cujo país com maior quantidade de publicações é os Estados Unidos.

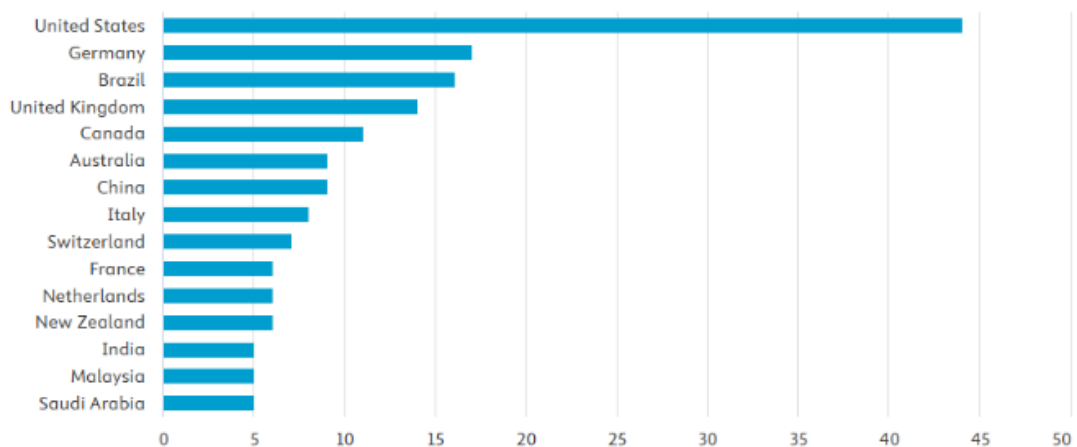


Figura 2 – Distribuição de artigos por país

Fonte: Scopus (2025).

Após a seleção, os materiais foram organizados por categorias temáticas e analisados com base em sua contribuição para a construção do referencial teórico e para o delineamento metodológico da pesquisa. Desse modo, foi utilizada a estratégia da análise de títulos e resumos dos artigos selecionados, para que os resultados possam ser auditáveis, replicáveis e úteis no suporte a pesquisas relacionadas a sistemas de captação de doadores de sangue, com o fim de que hospitais e hemocentros tenham estoques suficientes para atender às necessidades de pacientes que precisam de transfusões sanguíneas em diversas situações.

Sendo assim, a lista de artigos selecionados é apresentada no Quadro 4.

Tabela 4 – Lista de artigos selecionados

Título	Fonte
Blood donation support application: Contributions from experts on the tool's functionality	Da Silva; Praça Brasil; De Vasconcelos Filho; Brasil; Paiva; De Oliveira; Dos Santos (2021)
Effective methods for reactivating inactive blood donors: A stratified randomised controlled study	Ou-Yang; Bei; Liang; He; Chen; Fu (2020)
Evaluation of the Wateen App in the Blood-Donation Process in Saudi Arabia	Alessa (2022)
Exploring the Blood Donation Problem in the U.S. and the Use of Emergent and Disruptive Technologies to Attract New Donors	Keyser; Li; Kimber; Berman; Kerbage; Valero; Li; Ukuku (2025)
Factors associated with blood donation among college and university students in Wuhan, China: structural equation model	Ma; Yang; Gu; Ke; Du; Zheng (2024)
Mobile applications for encouraging blood donation: A systematic review and case study	Li; Valero; Keyser; Ukuku; Zheng (2023)
Profile of apheresis donors at Professor Alberto Antunes University Hospital of the Federal University of Alagoas	De Araújo Loureiro; De Lima Zielak; De Holanda Galvão Almeida; Barbosa; De Araújo (2021)
To donate or not donate, that is the question: An analysis of the critical factors of blood donation	Pereira; Sousa; De Matos; Rezende; Bueno; Dias (2016)
Voluntary Blood Donation in Modern Healthcare: Trends, Challenges, and Opportunities	Gasparovic Babic; Krsek; Baticic (2024)

Fonte: Elaborado pelos próprios autores (2025).

3.3 Resultados e discussões

A análise dos artigos selecionados permitiu identificar um conjunto relevante de soluções tecnológicas voltadas para a captação, o incentivo e a retenção de doadores de sangue, respondendo diretamente aos questionamentos levantados no protocolo de pesquisa. O levantamento revelou uma predominância de aplicativos móveis dedicados a facilitar a interação entre a população e os centros de saúde. Entre as plataformas destacadas na literatura, evidencia-se o Do-eSangue, desenvolvido com o intuito de apoiar a captação e fidelização ao integrar-se diretamente com a base de dados dos hemocentros, permitindo agendamentos e o acesso a informações personalizadas Silva et al. (2021). No cenário internacional, o Wateen, iniciativa

do Ministério da Saúde da Arábia Saudita, atua de forma semelhante no recrutamento e agendamento via dispositivos móveis Alessa (2022). Outras iniciativas relevantes incluem o Blood Hero, que explora técnicas de incentivo social para alavancar os estoques Li et al. (2023), e o Blood Note, focado em promover a doação por meio de testes preliminares de elegibilidade realizados diretamente pelo celular do usuário Li et al. (2023).

No que tange à aplicação de algoritmos de recomendação ou mecanismos inteligentes de sugestão de ações, os estudos apontam para o uso de filtros dinâmicos e cruzamento de dados como alternativas eficientes de conexão. O aplicativo Do-eSangue, por exemplo, realiza o disparo de convites segmentados com base no tipo sanguíneo, na localização geográfica e na necessidade de fenótipos raros. Embora o estudo não utilize a nomenclatura estrita de “algoritmos de recomendação”, essa filtragem avançada e baseada em critérios específicos de demanda atua, na prática, como uma recomendação direcionada de doadores Silva et al. (2021). Paralelamente, o Wateen demonstra um mecanismo ágil de correspondência ao notificar usuários específicos quando há demanda hospitalar por sua tipagem sanguínea, permitindo ainda que o próprio doador compartilhe a solicitação com sua rede de contatos, criando um fluxo orgânico e eficiente de recomendações Alessa (2022).

Além da infraestrutura técnica de conexão, os trabalhos mapeados evidenciam diversas funcionalidades e estratégias de incentivo essenciais para a promoção contínua da doação. A comunicação personalizada e direcionada surge como um fator crítico nesse cenário. Ou-Yang et al. (2020) demonstraram que, para reativar doadores inativos, o contato direto via chamadas telefônicas apresenta resultados significativamente superiores ao uso de SMS, pois viabiliza um diálogo individualizado capaz de resolver barreiras pessoais e motivar o retorno. Aliado a isso, o uso estratégico de redes sociais e a disponibilização de informações abrangentes integradas aos sistemas ajudam a combater mitos, esclarecer dúvidas frequentes e detalhar os critérios de elegibilidade, aumentando a confiança da população nos serviços de saúde.

Outro ponto de forte convergência entre as pesquisas é a adoção da gamificação e de recursos de fidelização. A implementação de sistemas de pontuação, medalhas virtuais (*badges*) e programas de fidelidade que garantem preferência no atendimento servem como um forte apelo de reconhecimento social do voluntário, incentivando a repetição do ato solidário.

Por fim, a literatura destaca que a otimização da experiência prática do doador é indispensável. Funcionalidades como o agendamento simplificado reduzem drasticamente o tempo de espera e as filas nos hemocentros, fatores que impactam diretamente na satisfação geral do usuário Alessa (2022). Quando essas plataformas permitem que o voluntário acompanhe seu histórico de doações e acesse laudos de exames de forma unificada, consolida-se um sentimento de propósito e cuidado com a própria saúde, evidenciando o potencial da tecnologia como ferramenta de transformação na hemoterapia.

3.4 Considerações

O mapeamento sistemático da literatura apresentado neste capítulo permitiu traçar um panorama claro sobre o uso da tecnologia como aliada na captação e retenção de doadores de sangue. Os resultados evidenciam que a adoção de sistemas computacionais — especialmente aplicativos móveis — deixou de ser uma inovação distante para se tornar uma necessidade estratégica na gestão hemoterápica. Funcionalidades como o envio de notificações segmentadas, o agendamento prévio e a aplicação de técnicas de gamificação provaram ser eficazes para engajar a população e reduzir os índices de desabastecimento nos estoques.

Apesar das contribuições valiosas encontradas na literatura, nota-se que grande parte dos estudos concentra sua análise nas interfaces com o usuário final e nas estratégias de marketing social, abordando de forma superficial a engenharia de software e a arquitetura estrutural que sustentam essas soluções. Existe uma lacuna no detalhamento de como as complexas regras de negócio desse domínio — que envolvem critérios médicos de elegibilidade, tipagem sanguínea e urgência hospitalar — são modeladas e processadas nos bastidores das aplicações.

Nesse sentido, os achados desta revisão validam a proposta central deste Trabalho de Conclusão de Curso. Ao constatar que a filtragem e a recomendação direcionada são os motores do sucesso das ferramentas atuais, reforça-se a relevância de uma API RESTful especializada, estruturada sobre DDD e princípios SOLID conforme discutido no Capítulo 2. O Capítulo 5 apresenta o estudo de caso, detalhando como os requisitos, a modelagem do domínio e a implementação da solução tratam, na prática, as lacunas de engenharia de software identificadas nesta revisão.

4 Métodos e recursos

Neste capítulo são descritos os procedimentos metodológicos adotados para o alcance dos objetivos propostos, abordando a abordagem de pesquisa escolhida, os instrumentos utilizados na coleta e análise de informações, as ferramentas empregadas no desenvolvimento da API e o cronograma de execução do trabalho.

4.1 Visão geral

Para o desenvolvimento da pesquisa adotou-se a abordagem qualitativa, que se destaca por enfatizar a análise de dados não numéricos, como documentos, normativas e literatura especializada, buscando compreender o significado e a complexidade dos fenômenos investigados.

A pesquisa qualitativa trabalha com o universo de significados, motivos, aspirações, crenças, valores e atitudes, o que corresponde a um espaço mais profundo das relações, dos processos e dos fenômenos que não pode ser reduzido à operacionalização de variáveis. Minayo (2001, p. 21)

Para a condução do trabalho, foram traçadas etapas sequenciais, ilustradas na Figura 3 e na Figura 4.

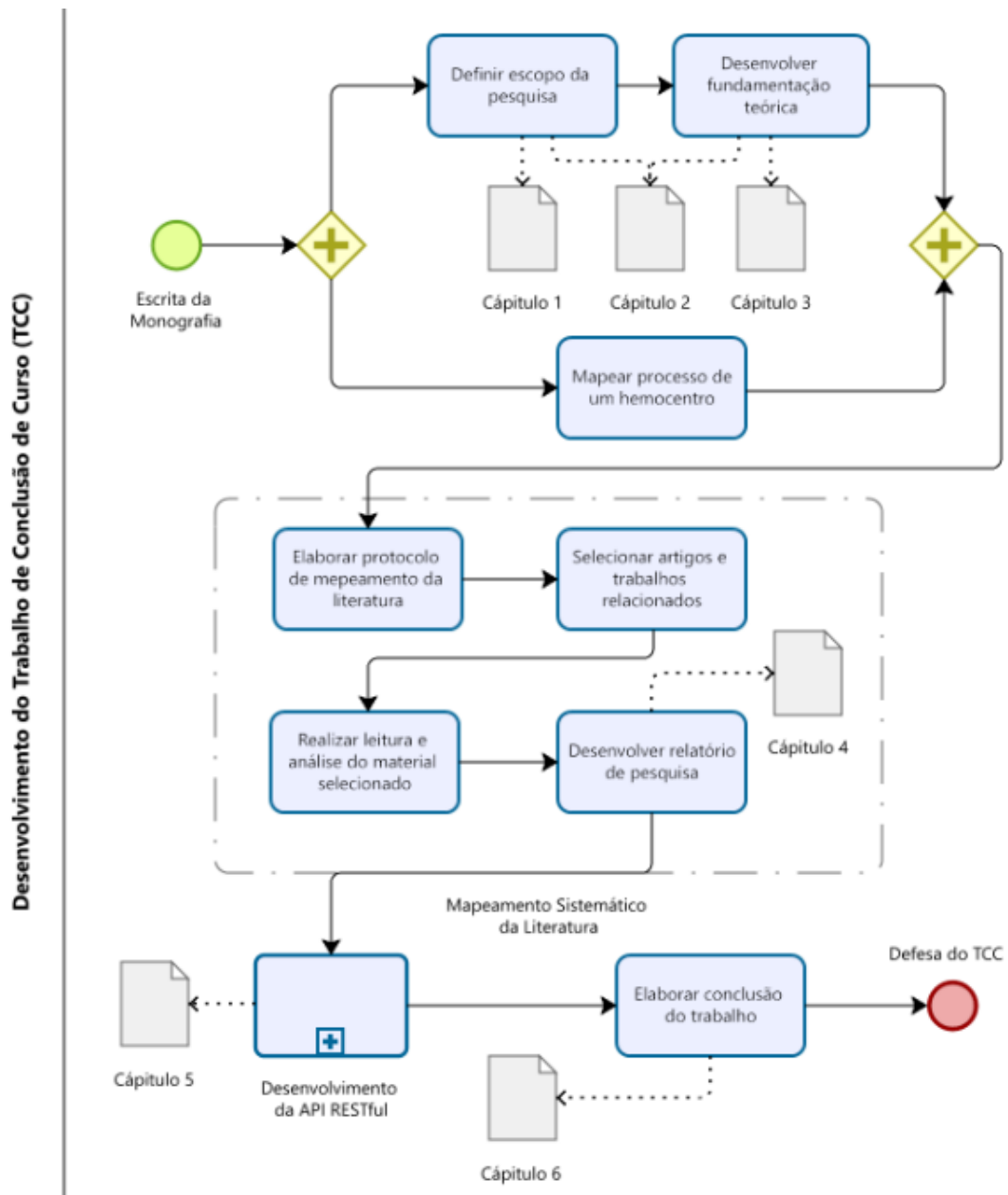


Figura 3 – Diagrama de processos do desenvolvimento do TCC

Fonte: Elaborado pelos próprios autores (2025).

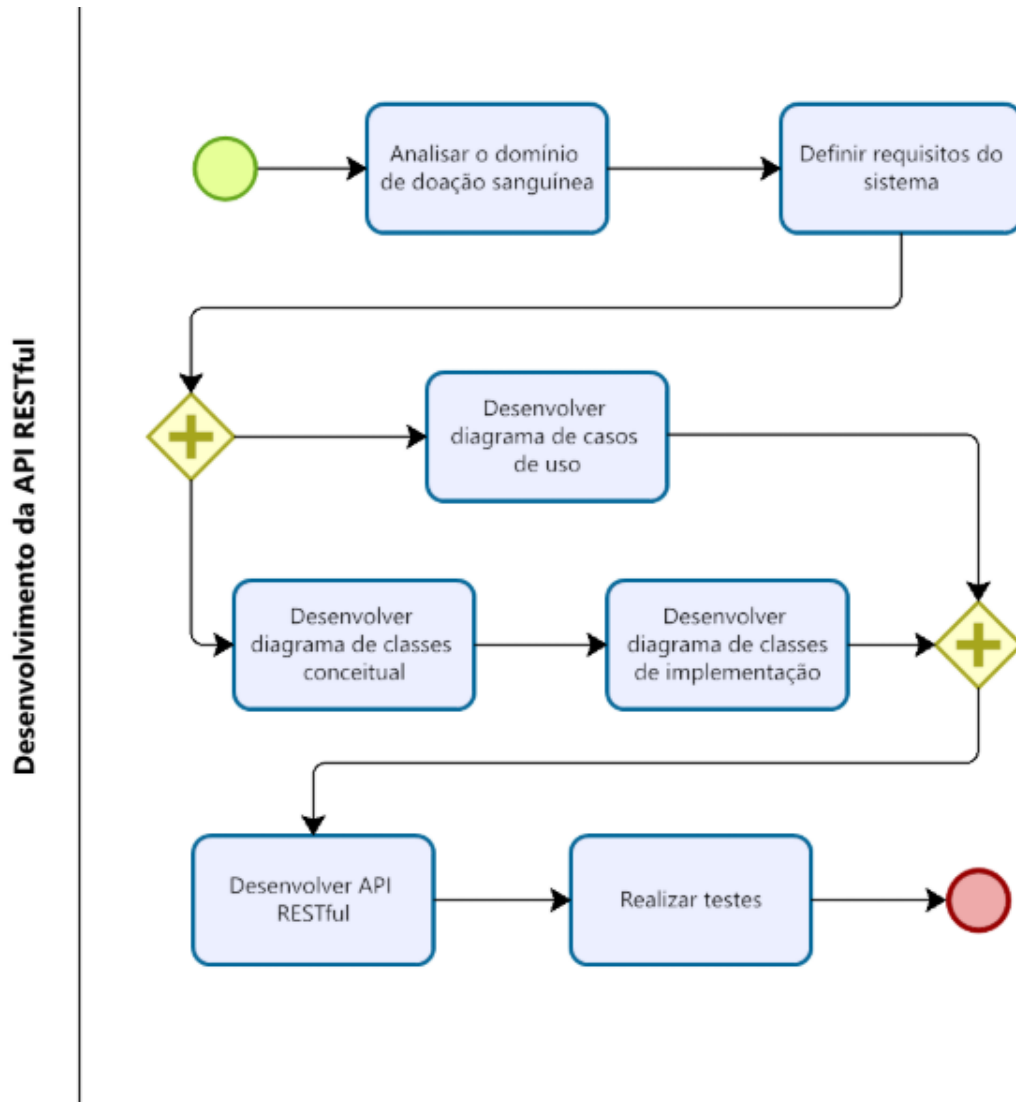


Figura 4 – Diagrama de processos do Desenvolvimento da API

Fonte: Elaborado pelos próprios autores (2025).

Inicialmente, definiu-se o escopo da pesquisa, o problema, as justificativas e os objetivos que fundamentam o Capítulo 1. Em seguida, conduziu-se o levantamento bibliográfico sobre OO, SOLID e DDD, aprofundados no Capítulo 2.

Paralelamente, realizou-se o mapeamento do processo de um hemocentro por meio de pesquisa documental e bibliográfica, buscando compreender o fluxo de trabalho, as regras de negócio e as necessidades do domínio da doação de sangue. Elaborou-se também um protocolo de mapeamento sistemático da literatura, cujos resultados são discutidos no Capítulo 3.

Com base no conhecimento adquirido nessas etapas, realizou-se a análise do domínio sob a ótica do DDD e a definição dos requisitos funcionais e não funcionais. Na sequência, elaborou-se o projeto da solução (casos de uso e diagramas de classes) e desenvolveu-se a API aplicando SOLID, com vistas à escalabilidade, manutenibilidade e testabilidade.

Por fim, os resultados foram avaliados por meio de testes unitários, seguidos da elabora-

ção das conclusões do trabalho.

4.2 Instrumentos e ferramentas

Para a coleta e análise dos dados, foram utilizados instrumentos compatíveis com os objetivos e a abordagem metodológica da pesquisa, orientados pela necessidade de obter informações confiáveis e contextualizadas sobre o cenário da doação de sangue e sobre as tecnologias empregadas no desenvolvimento da API.

4.2.1 Pesquisa exploratória-descritiva

A presente pesquisa caracteriza-se como exploratória-descritiva, abordagem que, segundo Gil (2008), proporciona maior familiaridade com o problema investigado e, simultaneamente, descreve as características do fenômeno estudado. Segundo Prodanov e Freitas (2013), essa combinação metodológica é especialmente adequada para estudos que envolvem questões sociais e de saúde pública, permitindo uma análise contextualizada e orientada para a intervenção.

O contexto de análise adotado foi o das normativas e do cenário de atuação dos hemocentros, com atenção às demandas da região de Campos dos Goytacazes, local de moradia e estudo dos pesquisadores. Essa escolha favoreceu maior familiaridade com os aspectos sociais, culturais e institucionais que envolvem a temática, além de facilitar o acesso a dados públicos e normativas das unidades hospitalares e hemocentros locais.

Segundo Pressman e Maxim (2016), o esforço inicial de compreensão do domínio e das necessidades reais dos *stakeholders* é indispensável para criar uma base sólida de requisitos e evitar o desenvolvimento de sistemas inadequados à realidade operacional. Nesse sentido, utilizou-se a análise documental e bibliográfica como técnica de coleta de dados, por permitir profundidade na obtenção de informações normativas e procedimentais a partir de manuais do Ministério da Saúde, portarias vigentes e literatura acadêmica.

Segundo Marconi e Lakatos (2003), a pesquisa documental é composta por etapas fundamentais — como a seleção, leitura crítica e categorização dos dados — que garantem a sistematização e a confiabilidade das informações obtidas. Sommerville (2011) reforça que a análise de documentos atua como instrumento de captura e construção do conhecimento, sendo essencial para a engenharia de requisitos em sistemas críticos, como os voltados à saúde pública.

O levantamento realizado a partir de documentos oficiais e estudos de caso sobre triagem, captação e gestão em hemocentros teve como objetivo compreender os fatores que influenciam a decisão de doar sangue, identificar barreiras e motivações entre potenciais doadores descritos na literatura, e subsidiar os requisitos da API proposta.

4.2.2 Desenvolvimento da API

O desenvolvimento da API foi orientado por boas práticas de engenharia de *software*, com foco na escalabilidade, na manutenibilidade e na clareza estrutural do código. Adotou-se DDD e SOLID como diretrizes arquiteturais (Seções 2.3 e 2.2). A camada de segurança contempla autenticação baseada em *JSON Web Token* (JWT), padrão amplamente adotado em APIs RESTful para autenticação sem estado (*stateless*).

4.2.2.1 Linguagem de programação (Java)

A linguagem escolhida para a implementação foi o Java, em sua versão 17 (*Long Term Support* — LTS). Java é uma linguagem orientada a objetos, fortemente tipada e amplamente documentada na literatura Deitel e Deitel (2017). A tipagem estática do Java favorece a implementação de *Value Objects* e *Entities*, garantindo que erros de domínio sejam capturados em tempo de compilação. Em contrapartida, uma crítica comum ao Java é a sua verbosidade e o consumo de memória da *Java Virtual Machine* (JVM), se comparado a linguagens como Go ou Python, conforme discutido por Bloch (2018). Ainda assim, a robustez da linguagem compensa essas restrições para o contexto corporativo.

4.2.2.2 *Framework* de desenvolvimento (Spring Boot)

Para agilizar o desenvolvimento e a configuração da infraestrutura, utilizou-se o *framework* Spring Boot. Segundo Walls (2019), o Spring Boot aplica o conceito de “convenção sobre configuração”, permitindo que o desenvolvedor foque na lógica de negócio enquanto o *framework* gerencia a injeção de dependência e a conexão com o banco de dados. Por outro lado, a facilidade do Spring pode esconder a complexidade subjacente, dificultando o *debug* (depuração) caso o desenvolvedor não entenda o ciclo de vida dos *beans* gerenciados pelo *container*.

4.2.2.3 Ambiente de desenvolvimento e versionamento (Visual Studio Code e Git)

Para a codificação, utilizou-se o editor de código Visual Studio Code (VS Code), devido ao seu vasto ecossistema de extensões para Java e suporte nativo a terminais leves. Para o controle de versão, adotou-se o Git, hospedando o repositório na plataforma GitHub. Segundo Chacon e Straub (2014), o versionamento distribuído é essencial para manter o histórico de alterações seguro e permitir ramificações (*branches*) para testes de novas funcionalidades sem comprometer o código estável (*main*).

4.3 Cronograma

Tabela 5 – Cronograma de execução da primeira etapa do TCC

Atividade	Ago/25	Set/25	Out/25	Nov/25	Dez/25	Jan/26
Definição do escopo, problema e objetivos	X	X				
Fundamentação teórica (OO, SOLID, DDD)		X	X	X		
Pesquisa documental do domínio hemoterápico		X	X			
Protocolo e mapeamento sistemático da literatura			X	X	X	
Análise de domínio com DDD				X	X	X
Levantamento de requisitos (RF/RNF)					X	X
Redação dos Capítulos 1 a 4	X	X	X	X	X	X

Fonte: Elaborado pelos próprios autores (2025).

Tabela 6 – Cronograma de execução da segunda etapa do TCC

Atividade	Fev/26	Mar/26	Abr/26	Mai/26	Jun/26	Jul/26
Modelagem UML (casos de uso e classes)	X	X				
Implementação da API (Java / Spring Boot)		X	X	X		
Persistência (MongoDB) e autenticação (JWT)			X	X		
Motor de recomendação de doadores			X	X	X	
Testes unitários e de integração				X	X	
Avaliação da proposta e análise de resultados					X	X
Redação final (Capítulos 5 a 7) e revisão	X	X	X	X	X	X

Fonte: Elaborado pelos próprios autores (2025).

5 Estudo de caso: requisitos, modelagem e implementação

5.1 Visão geral da solução

Este capítulo apresenta a solução tecnológica desenvolvida para apoiar o processo de captação e gerenciamento de solicitações de doação de sangue. A proposta materializa-se na construção de uma API RESTful focada exclusivamente na camada de *backend*, responsável pelo gerenciamento de participantes (pessoas e organizações), solicitações de doação, registro do histórico de doadores e recomendação de solicitações compatíveis.

O desenvolvimento foi orientado pelos princípios do DDD e pelos princípios SOLID, já fundamentados no Capítulo 2. Essa abordagem permitiu que a estrutura do software refletisse a realidade do domínio da hemoterapia, reduzindo o acoplamento entre os componentes e facilitando a evolução contínua do sistema. As seções seguintes detalham o levantamento de requisitos, a modelagem do sistema, a organização do domínio e da arquitetura, a implementação técnica e, por fim, a estratégia de recomendação e de cumprimento de metas, que constitui o núcleo funcional da API.

5.2 Levantamento de requisitos

A extração de requisitos ocorreu a partir da análise documental e bibliográfica descrita no Capítulo 4, alinhando as necessidades dos diferentes atores envolvidos no processo de doação de sangue às capacidades técnicas do sistema proposto.

5.2.1 Requisitos funcionais

Os requisitos funcionais representam as operações que a API disponibiliza para atender às rotinas de cadastro, gerenciamento de solicitações, recomendação e controle de doações. O Quadro 7 sintetiza as funcionalidades identificadas e implementadas, refletindo diretamente os recursos (*endpoints*) expostos pela API.

Tabela 7 – Requisitos funcionais

Código	Descrição
RF01	Permitir o cadastro de pessoas no sistema.
RF02	Permitir o cadastro de organizações no sistema.
RF03	Permitir que uma pessoa cadastrada registre um perfil de doador.
RF04	Permitir que uma pessoa ou organização registre um perfil de requisitante.
RF05	Permitir que uma organização cadastrada registre um perfil de banco de sangue (hemocentro).
RF06	Permitir a autenticação de usuários por meio de credenciais válidas, emitindo um JWT.
RF07	Permitir a atualização do nome cadastral de um participante (<i>party</i>).
RF08	Permitir a atualização do perfil do doador.
RF09	Permitir que requisitantes criem solicitações de doação de sangue.
RF10	Permitir a atualização da meta de bolsas de sangue de uma solicitação.
RF11	Permitir a atualização do prazo (data-limite) de uma solicitação.
RF12	Permitir o cancelamento de uma solicitação de doação.
RF13	Permitir a consulta das solicitações de doação de um participante.
RF14	Permitir a consulta das solicitações de doação recomendadas para um doador.
RF15	Permitir a notificação de doadores potenciais elegíveis para uma solicitação.
RF16	Permitir o registro de uma doação pendente.
RF17	Permitir o reagendamento de uma doação pendente.
RF18	Permitir a confirmação (conclusão) de uma doação pendente.
RF19	Permitir o registro direto de uma doação já concluída.
RF20	Permitir a consulta do histórico de doações de um doador.
RF21	Permitir a consulta do resumo (<i>summary</i>) de um doador.
RF22	Permitir o ajuste da distância máxima considerada nas recomendações de um doador.

Fonte: Elaborado pelos próprios autores (2026).

5.2.2 Requisitos não funcionais

Além das funcionalidades diretas, foram definidos requisitos não funcionais para estabelecer os parâmetros de qualidade, segurança e arquitetura esperados. Considerando a adoção do DDD e da arquitetura em camadas, o Quadro 8 apresenta as diretrizes técnicas estabelecidas para a solução.

Tabela 8 – Requisitos não funcionais

Código	Descrição
RNF01	O sistema deve disponibilizar seus serviços por meio de uma API RESTful utilizando o protocolo HTTP.
RNF02	O sistema deve autenticar usuários por meio de JWT.
RNF03	O sistema deve persistir os dados em banco de dados NoSQL (MongoDB).
RNF04	O sistema deve permitir evolução e manutenção sem impacto significativo nas demais camadas da aplicação.
RNF05	O sistema deve seguir uma arquitetura em camadas orientada pelos princípios do DDD.
RNF06	O sistema deve aplicar os princípios SOLID para favorecer baixo acoplamento e alta coesão.
RNF07	A API deve retornar respostas em formato JSON.
RNF08	O sistema deve validar os dados recebidos antes do processamento das operações.
RNF09	O sistema deve controlar o acesso às funcionalidades protegidas por meio de autorização baseada em perfis (<i>roles</i>) de usuário.
RNF10	O sistema deve preservar a consistência de escritas concorrentes sobre uma mesma solicitação de doação, evitando a perda de atualizações.

Fonte: Elaborado pelos próprios autores (2026).

5.3 Modelagem do sistema

Após a consolidação dos requisitos, procedeu-se com a modelagem do sistema para descrever as interações entre os atores e a API, permitindo visualizar o comportamento esperado da solução sob a perspectiva estrutural e de uso.

5.3.1 Casos de uso

A modelagem de casos de uso, em notação UML, sintetiza as operações disponíveis para os diferentes perfis de acesso. O Quadro 9 apresenta o mapeamento dos casos de uso e seus respectivos atores, incluindo tanto as operações representadas nos diagramas a seguir quanto operações complementares oferecidas pela API para os mesmos atores.

Tabela 9 – Lista de casos de uso

Código	Caso de uso	Ator principal
UC01	Cadastrar-se como pessoa (<i>Sign Up as Person</i>)	Guest
UC02	Cadastrar-se como organização (<i>Sign Up as Organization</i>)	Guest
UC03	Autenticar (<i>Authenticate</i>)	Guest
UC04	Registrar perfil de doador	Person
UC05	Registrar perfil de requisitante	Person / Organization
UC06	Registrar perfil de banco de sangue	Organization
UC07	Atualizar dados cadastrais / perfil do doador	Donor / Requester
UC08	Registrar solicitação de doação (<i>Register Donation Request</i>)	Requester
UC09	Consultar recomendações de solicitações (<i>Consult Recommendations</i>)	Donor
UC10	Registrar doação pendente (<i>Register Pending Donation</i>)	Donor
UC11	Registrar doação realizada (<i>Register Completed Donation</i>)	Donor
UC12	Completar doação pendente (<i>Complete a Pending Donation</i>)	Donor
UC13	Consultar histórico de doações (<i>See Donation History</i>)	Donor
UC14	Gerenciar solicitações — atualizar meta/prazo, cancelar e consultar	Requester
UC15	Atualizar distância máxima para recomendações	Donor
UC16	Notificar doadores potenciais elegíveis	Requester
UC17	Reagendar doação pendente	Donor
UC18	Consultar resumo do doador (<i>Donor Summary</i>)	Donor

Fonte: Elaborado pelos próprios autores (2026).

A representação gráfica dessas interações é dividida de acordo com o nível de acesso dos atores:

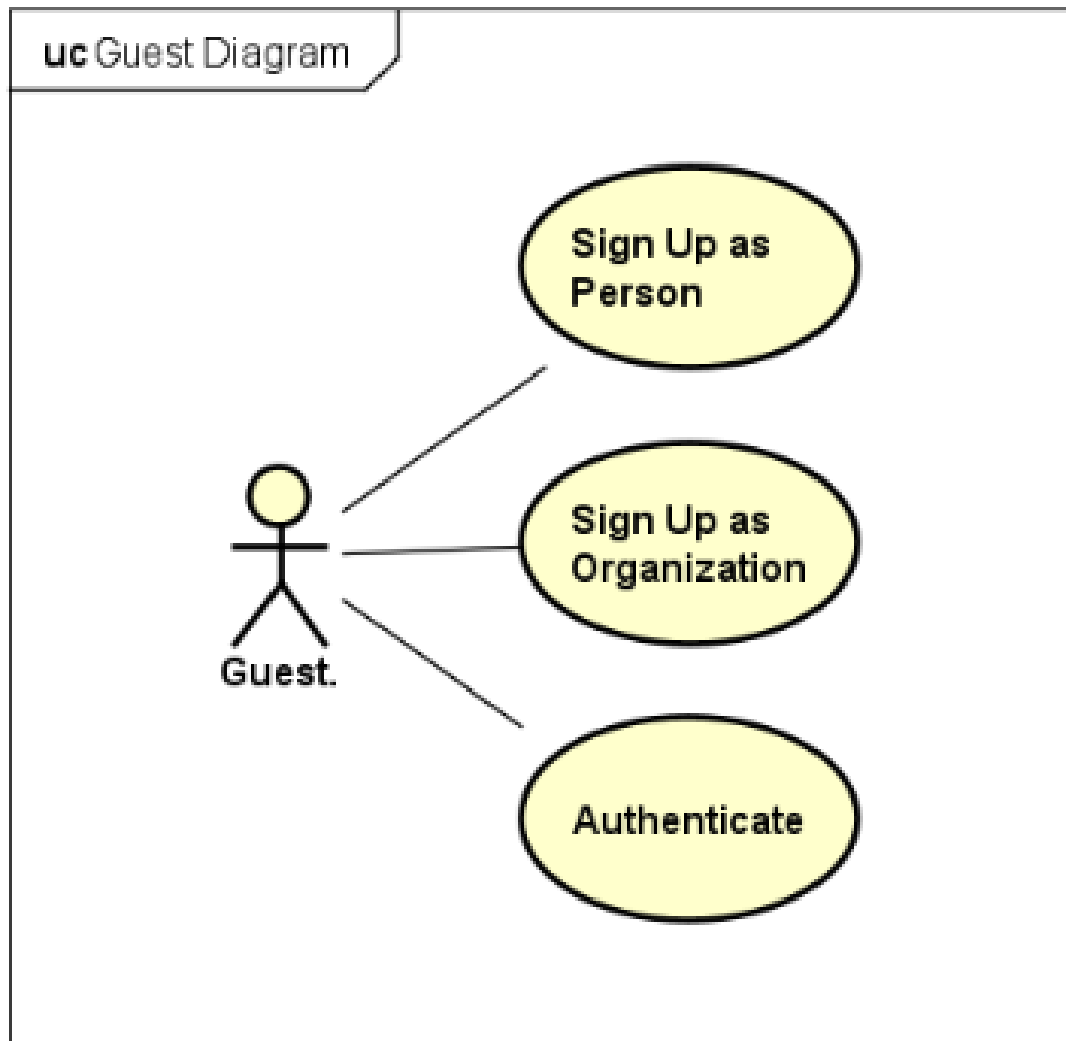


Figura 5 – Diagrama de caso de uso de visitante

Fonte: Elaborado pelos próprios autores (2026).

O ator *Guest* representa utilizadores sem cadastro prévio, limitados às operações de criação de contas e autenticação (UC01, UC02 e UC03).

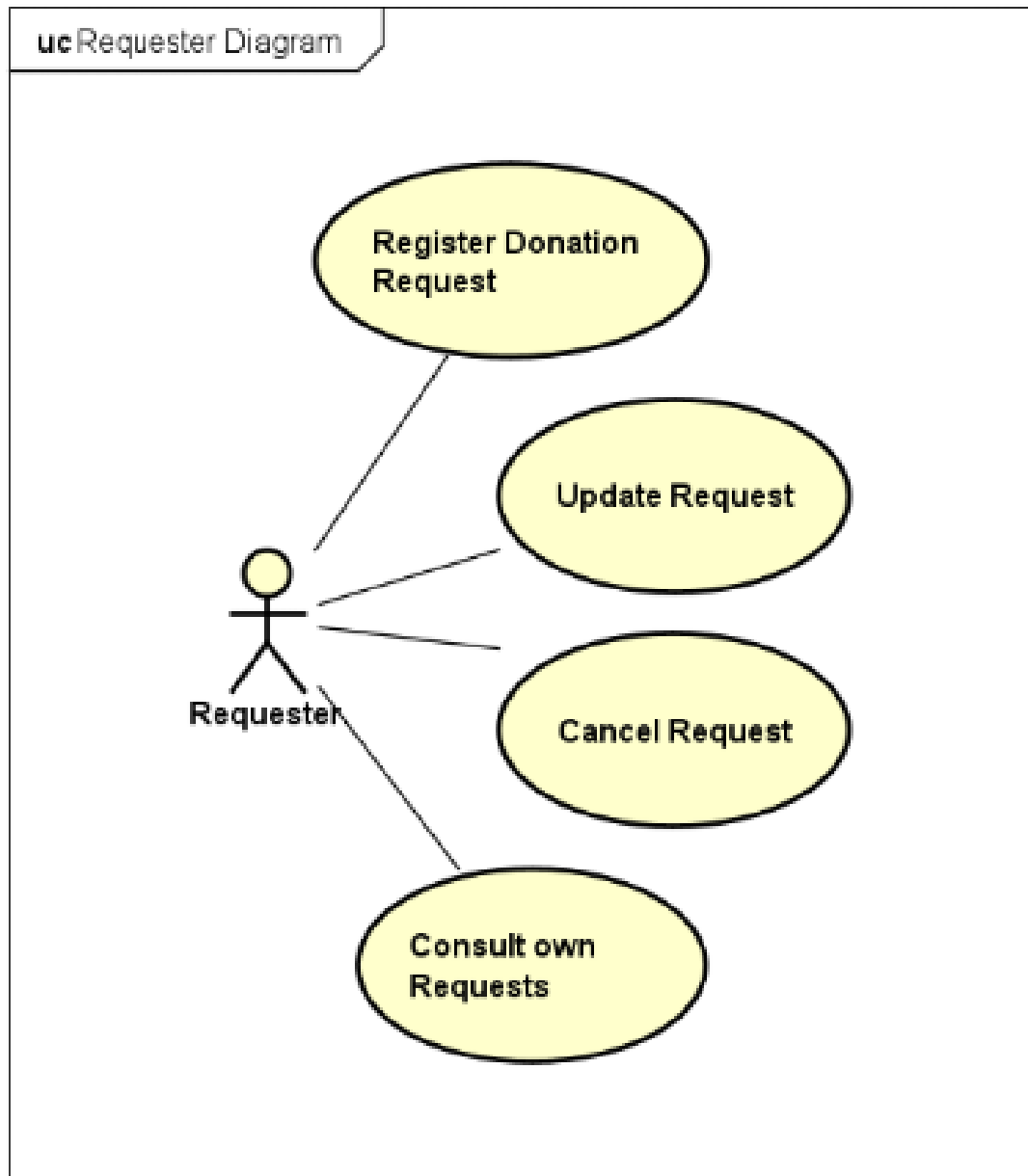


Figura 6 – Diagrama de caso de uso de requisitante

Fonte: Elaborado pelos próprios autores (2026).

O ator *Requester* concentra as permissões relacionadas à criação e à gestão de solicitações de sangue (UC08 e UC14), além do disparo de notificações a doadores potenciais quando uma solicitação precisa de maior alcance (UC16).

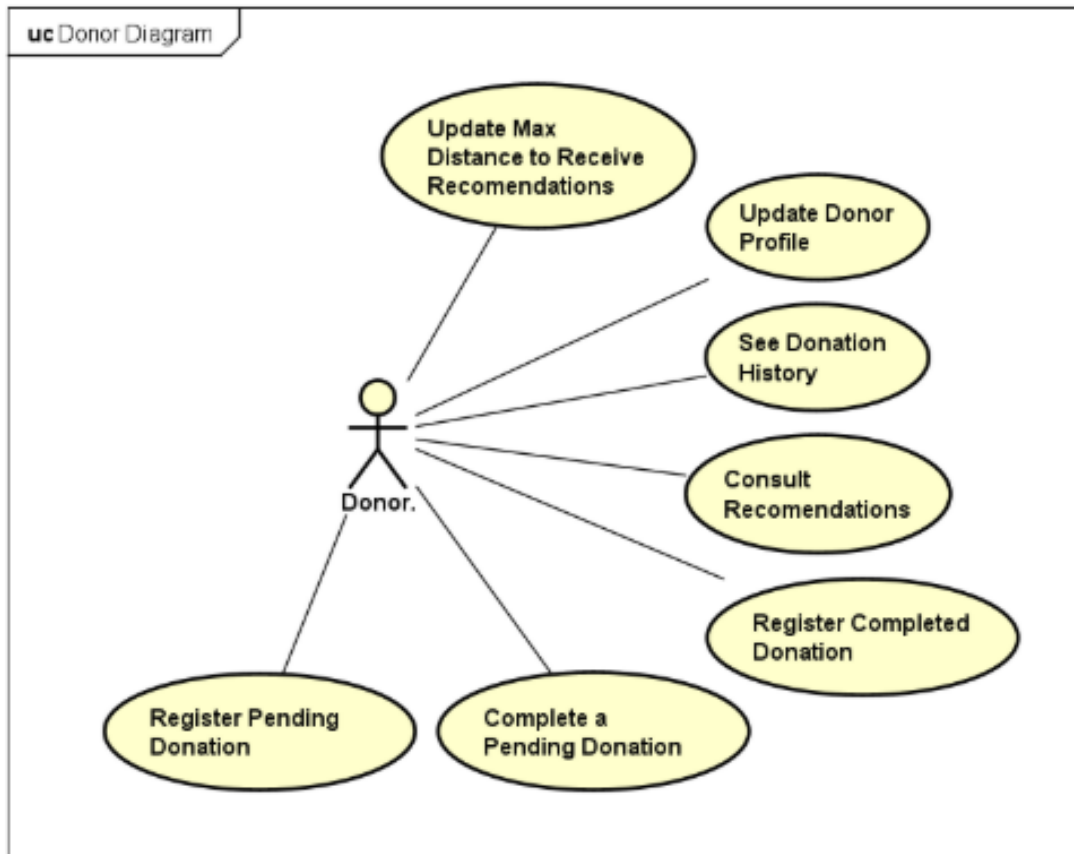


Figura 7 – Diagrama de caso de uso de doador

Fonte: Elaborado pelos próprios autores (2026).

O ator *Donor* possui acesso ao núcleo do processo de doação, incluindo a consulta às solicitações recomendadas, o ajuste da distância máxima de recomendação, a gestão do próprio histórico hemoterápico e a consulta ao seu resumo (UC09, UC10, UC11, UC12, UC13, UC15, UC17 e UC18).

5.3.2 Diagramas de classes

Para representar a estrutura estática da aplicação e evidenciar a aplicação do DDD na prática — cujos conceitos de entidade, agregado e objeto de valor já foram apresentados no Capítulo 2 —, a modelagem de classes foi concebida em duas perspectivas distintas: conceitual e de implementação. A perspectiva conceitual mapeia o vocabulário do negócio e as relações semânticas do mundo real, omitindo métodos e tipos de dados para facilitar a validação com especialistas. Em contrapartida, a perspectiva de implementação atua como uma representação estrutural do código-fonte construído, detalhando os limites transacionais, o encapsulamento e as diretrizes arquiteturais.

A Figura 8 apresenta o diagrama de classes conceitual focado nos participantes e em seus papéis (*Roles*).

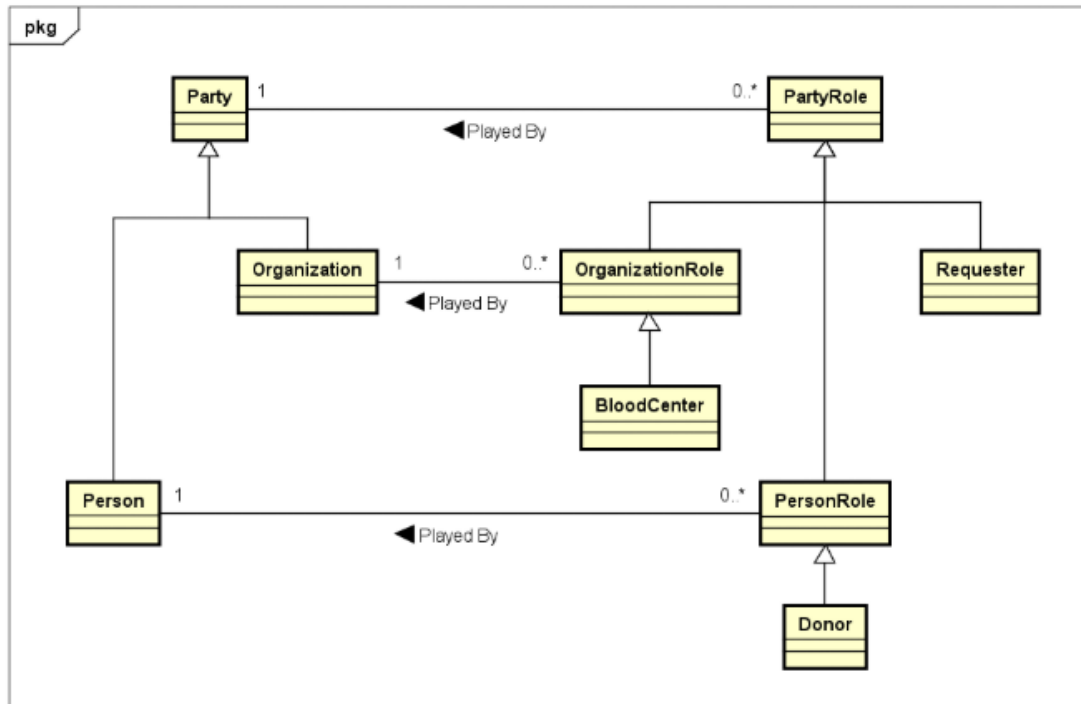


Figura 8 – Diagrama conceitual dos participantes

Fonte: Elaborado pelos próprios autores (2026).

Observa-se a utilização da abstração central **Party**, que atua como classe base para **Person** (Pessoa) e **Organization** (Organização). Essa modelagem reflete a flexibilidade do domínio, permitindo que as entidades assumam múltiplos papéis, como **Donor** (Doador) e **Requester** (Requisitante), eliminando a duplicidade de dados no sistema.

Dando prosseguimento à representação do negócio, a Figura 9 ilustra o diagrama de classes conceitual do processo de doação.

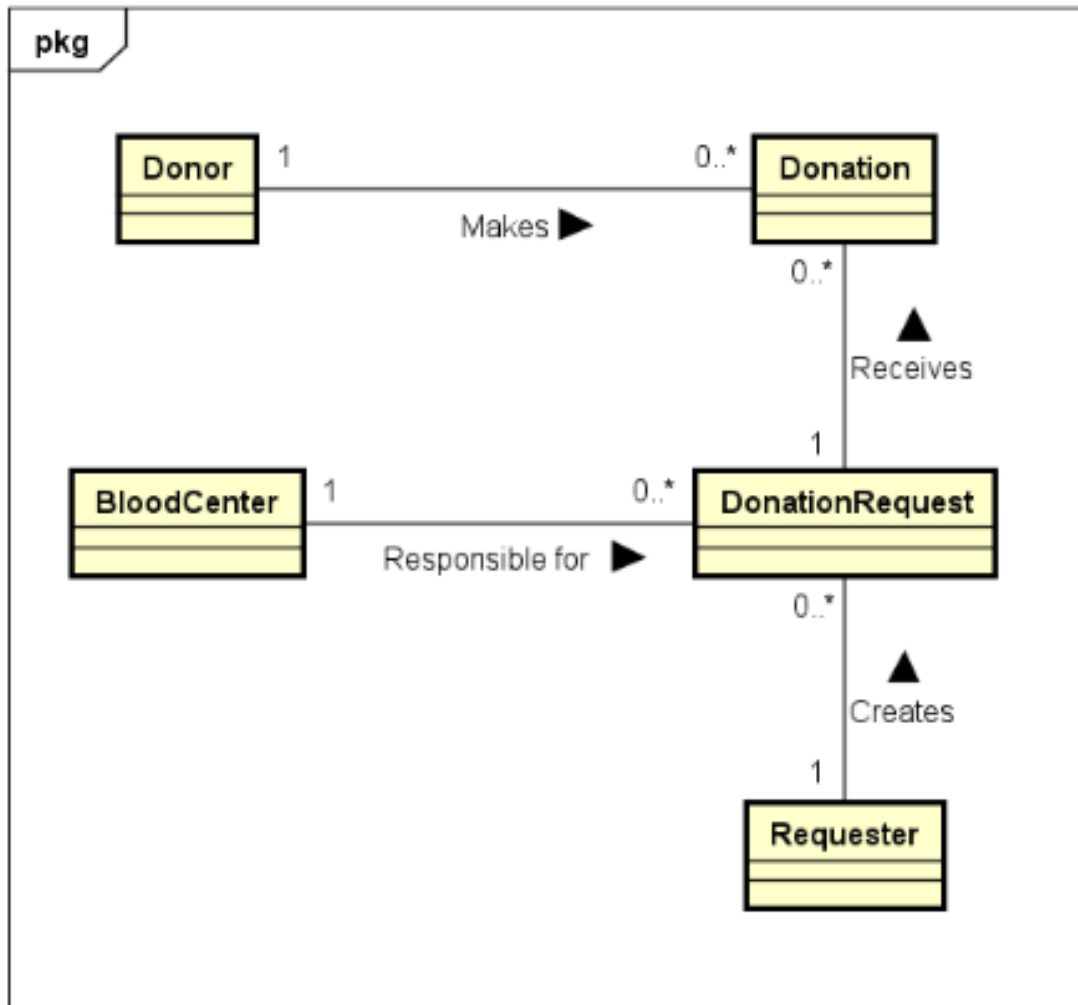


Figura 9 – Diagrama conceitual do processo de doação

Fonte: Elaborado pelos próprios autores (2026).

Neste nível de abstração, evidencia-se a associação semântica entre a solicitação (**DonationRequest**) e o ato de doar (**Donation**). O fluxo natural demonstra que um **Requester** origina a solicitação, que é vinculada a um **BloodCenter** e, posteriormente, recebe as doações efetivadas por um **Donor**.

Elevando o rigor para o design de software, a Figura 10 detalha a implementação técnica do agregado **DonationRequest**. A classe é modelada como uma raiz de agregado («AggregateRoot»), atuando como a única porta de entrada para a proteção de invariantes, como as datas-limite e a urgência da solicitação. Observa-se a aplicação da herança por meio da superclasse abstrata **DomainObject**, responsável por compor os identificadores únicos globais («ValueObject» **DomainID**). Destaca-se também a dependência tracejada («use») com a classe **Donation**, a qual evidencia o baixo acoplamento da arquitetura: a requisição não armazena doações em memória, apenas as avalia de forma transitória por meio de parâmetros no método **acceptsDonation()**.

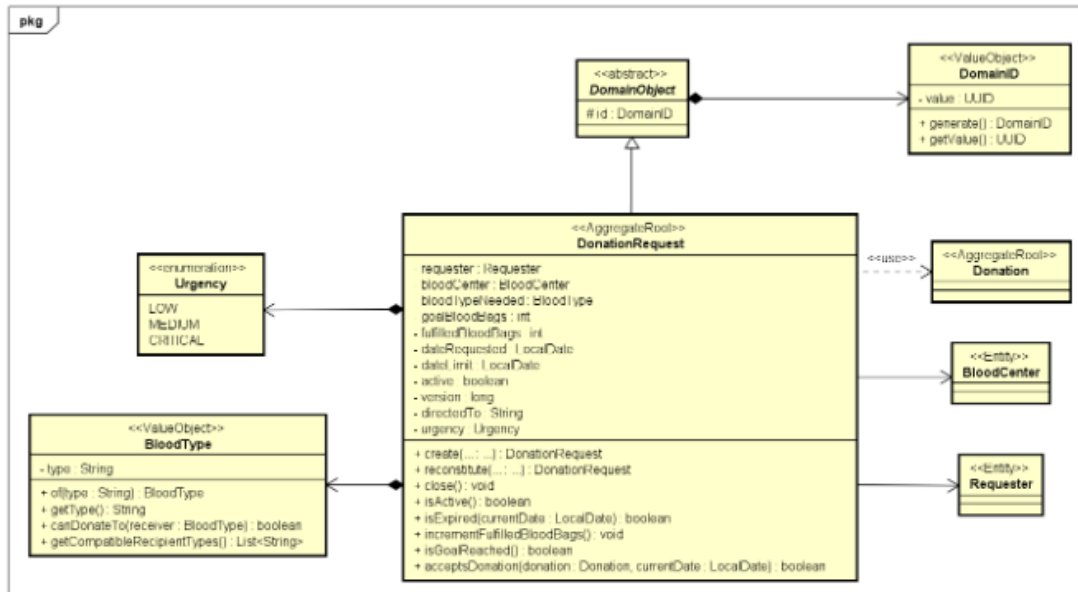


Figura 10 – Diagrama de implementação: agregado DonationRequest

Fonte: Elaborado pelos próprios autores (2026).

Por fim, a Figura 11 apresenta o diagrama de implementação do agregado **Donation**. Tratada também como uma raiz de agregado autônoma, a classe gerencia seu próprio ciclo de vida de estados (pendente, concluída, cancelada). A navegabilidade unidirecional explícita demonstra que a doação possui a referência imediata do Donor e do BloodCenter envolvidos, assegurando a consistência da operação de doação sem a necessidade de carregar listas e metadados completos de uma solicitação simultaneamente.

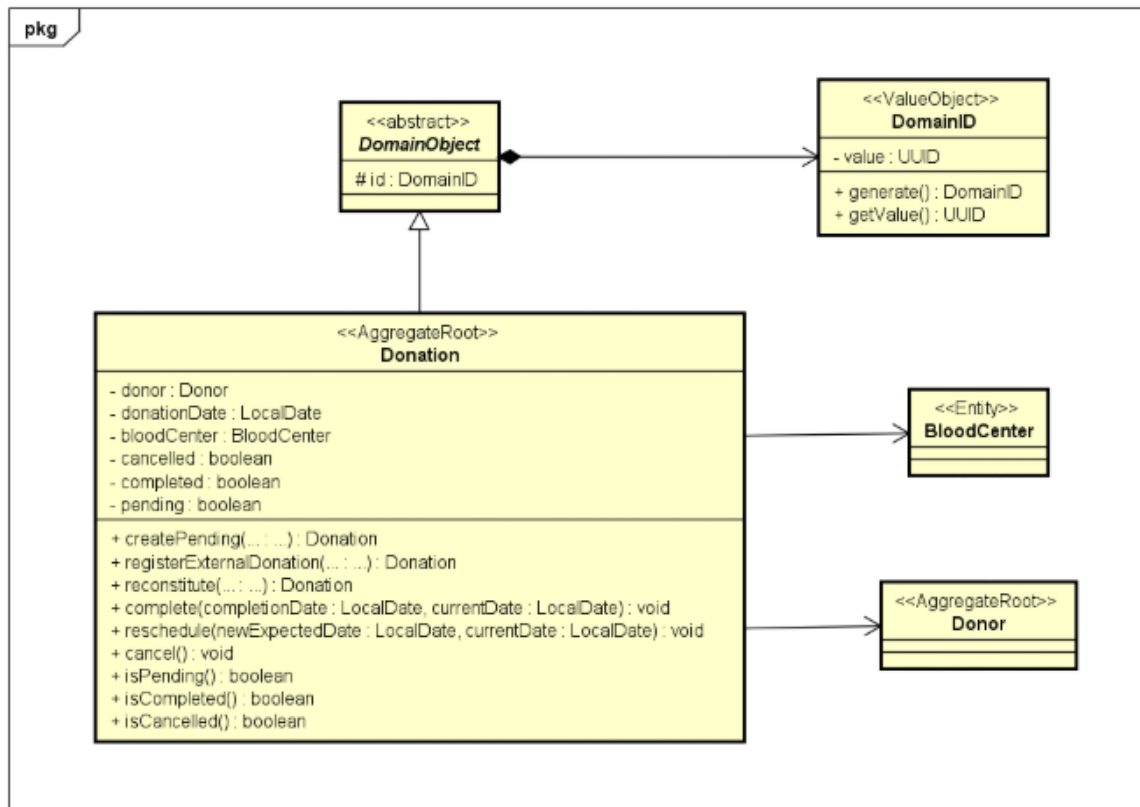


Figura 11 – Diagrama de implementação: agregado Donation

Fonte: Elaborado pelos próprios autores (2026).

5.4 Domínio e arquitetura

A partir das interações mapeadas, estruturou-se o núcleo da aplicação: o modelo de domínio e a organização das camadas que o protegem de dependências tecnológicas externas.

5.4.1 Linguagem ubíqua e organização em camadas

Conforme estabelecido nos fundamentos do DDD (Capítulo 2), a modelagem foi construída sobre uma linguagem ubíqua, garantindo que termos como *DonationRequest*, *Donor*, *Requester* e *BloodCenter* transitem com a mesma semântica tanto nas discussões de projeto quanto no código-fonte final. Essa padronização reduz ambiguidades entre a gestão de perfis de participantes e o ciclo de vida das doações. Cabe ressaltar que essa separação é tratada aqui como uma interpretação de projeto inspirada na noção de contexto delimitado (*Bounded Context*), e não como uma divisão formal em módulos ou serviços independentes: o código-fonte é organizado fisicamente em um único pacote de domínio, sendo a fronteira entre as duas áreas conceituais reforçada por convenção de nomenclatura e coesão de classes, não por isolamento físico.

Do ponto de vista estrutural, o código é organizado nos seguintes pacotes, cada um com uma responsabilidade específica:

- **domain:** entidades, objetos de valor e regras de negócio, livres de dependências de frameworks;
- **application/usecase:** orquestração dos casos de uso, coordenando chamadas ao domínio e aos repositórios;
- **interfaces/rest:** *controllers* e DTOs responsáveis pela exposição HTTP da API;
- **infra/persistence:** repositórios e *schemas* de persistência em MongoDB;
- **infra/security:** filtros de autenticação/autorização baseados em JWT.

O fluxo padrão de uma chamada percorre essas camadas em uma única direção — `interfaces/rest` → `application/usecase` → `domain` → `infra/persistence` —, de modo que a camada de domínio nunca dependa diretamente de detalhes de infraestrutura ou de protocolo HTTP, conforme recomendado por Evans (2003) e reforçado por Martin (2008) quanto à inversão de dependência entre módulos de alto e baixo nível.

5.4.2 Entidades, agregados e objetos de valor

A estrutura central do domínio foi modelada de forma hierárquica e orientada a objetos. O Quadro 10 sintetiza as principais entidades e agregados responsáveis por garantir a consistência das transações no sistema.

Tabela 10 – Entidades e suas responsabilidades

Entidade	Responsabilidade
Party	Abstrai características comuns dos participantes do sistema.
Person	Representa pessoas físicas cadastradas.
Organization	Representa instituições cadastradas.
Donor	Representa o perfil de doador e suas regras de elegibilidade.
Requester	Representa o perfil requisitante.
BloodCenter	Representa o perfil de banco de sangue vinculado a uma organização.
DonationRequest	Representa uma solicitação de doação de sangue, incluindo sua meta e seu progresso.
Donation	Representa uma doação registrada no sistema.

Fonte: Elaborado pelos próprios autores (2026).

A entidade `Party` foi adotada como abstração principal da hierarquia, aplicando herança para derivar `Person` e `Organization`. A partir dessas bases, o sistema utiliza polimorfismo

para permitir que os participantes assumam papéis (*Roles*) específicos, como *Donor*, *Requester* ou *BloodCenter*, dependendo de suas ações no sistema.

As entidades *DonationRequest* e *Donation* operam como raízes de agregados (*Aggregate Roots*), concentrando regras de negócio — como a validação de datas-limite e a verificação de compatibilidade sanguínea — e protegendo o estado interno dos objetos por meio do encapsulamento. Adicionalmente, conceitos imutáveis sem identidade própria, como *Address* (Endereço), *BloodType* (Tipo Sanguíneo) e *DomainID*, foram modelados como objetos de valor, na linha proposta por Evans (2003) e detalhada por Vernon (2013), aumentando a segurança e a previsibilidade do modelo.

5.5 Implementação

5.5.1 Persistência de dados

A persistência foi implementada sobre MongoDB, um banco de dados NoSQL orientado a documentos. Cada agregado de domínio possui um *schema* de persistência correspondente (por exemplo, *DonationRequestSchema* e *DonationSchema*), responsável por converter o objeto de domínio em um documento e vice-versa por meio de métodos dedicados de mapeamento (construtor a partir do domínio e *toDomain(...)*). Essa separação evita que anotações e particularidades do banco de dados vazem para as classes de domínio, mantendo o isolamento preconizado no DDD.

Um ponto de destaque é o controle de concorrência da solicitação de doação: o *DonationRequestSchema* utiliza controle de versão otimista (anotação *@Version*), de forma que duas escritas concorrentes sobre a mesma solicitação — por exemplo, um recálculo de cumprimento de meta e uma atualização manual de prazo — sejam detectadas pelo MongoDB. Ao identificar um conflito de versão, o repositório converte a exceção de persistência em uma *IllegalStateException*, sinalizando de forma explícita ao caso de uso que a operação deve ser reavaliada. Além disso, índices compostos foram definidos sobre campos como situação ativa, tipo sanguíneo e data-limite, bem como um índice geoespacial sobre a localização do hemocentro, otimizando as consultas de recomendação descritas na Seção 5.6.

5.5.2 Segurança e autenticação

O controle de acesso é implementado por meio de autenticação baseada em JWT. O endpoint público de login (*/auth/login*) valida as credenciais do usuário e emite um *token* assinado, que passa a ser exigido nas demais chamadas autenticadas por meio de um filtro (*JwtAuthenticationFilter*) na camada *infra/security*. Esse filtro intercepta as requisições antes de chegarem aos *controllers*, valida o *token* e popula o contexto de segurança com a identidade do participante autenticado, permitindo que os casos de uso apliquem regras de auto-

rização — como a verificação de que apenas o próprio requisitante pode gerenciar ou notificar doadores sobre uma solicitação de sua autoria.

5.5.3 Serviços externos e documentação da API

A notificação de doadores é abstraída por uma interface de domínio (`NotificationServiceInterface`) permitindo múltiplas implementações na camada de infraestrutura — como uma implementação por e-mail e uma implementação simplificada para ambiente de desenvolvimento — sem que o caso de uso que dispara a notificação precise conhecer o mecanismo concreto de envio. Essa inversão de dependência, alinhada aos princípios SOLID discutidos no Capítulo 2, evita que o domínio fique acoplado a bibliotecas de terceiros.

Para a documentação e a exploração interativa dos recursos da API, foi integrada a biblioteca *springdoc-openapi*, que gera automaticamente a especificação OpenAPI a partir dos *controllers* e disponibiliza uma interface de teste dos *endpoints*. Essa documentação viva reduz o custo de manutenção da documentação técnica e facilita a validação manual dos fluxos descritos neste capítulo.

5.6 Recomendação de solicitações e cumprimento de metas

Esta seção detalha o mecanismo central da API: a recomendação de solicitações compatíveis a doadores, a notificação de doadores potenciais e o acompanhamento do cumprimento de metas de cada solicitação. Ao contrário de um simples contador incrementado a cada doação, o sistema trata o cumprimento de metas como o resultado de um algoritmo de alocação, sincronizado sempre que uma doação é concluída.

5.6.1 Recomendação de solicitações para o doador

O caminho de leitura é implementado pelo `GetRecommendedRequestsUseCase`, acionado pelo *endpoint* `GET /donation-requests/recommendations`. Dado um doador autenticado, o caso de uso primeiro verifica sua elegibilidade para doar na data corrente (`Donor.isEligibleToDonate`), considerando faixa etária e intervalo mínimo desde a última doação. Em seguida, busca no repositório as solicitações ativas compatíveis com o tipo sanguíneo do doador, dentro da distância máxima de recomendação configurada por ele (RF22) e ainda não expiradas, aproveitando o índice geoespacial descrito na Seção anterior.

As solicitações candidatas que já atingiram a meta (`isGoalReached()`) são descartadas, e as demais são ordenadas por proximidade geográfica, urgência e proximidade da data-limite, de modo que o doador visualize primeiro as solicitações mais próximas, mais urgentes e mais próximas de expirar. O resultado retornado inclui, para cada solicitação recomendada, a distância estimada, a urgência, a meta de bolsas e o total já cumprido — este último lido diretamente do campo persistido `fulfilledBloodBags`, sem qualquer recálculo nesse momento.

5.6.2 Notificação de doadores potenciais

Complementarmente à recomendação, o requisitante pode disparar notificações ativas por meio do *endpoint* `POST /donation-requests/{id}/notify`, implementado pelo `NotifyPotentialDonorsUseCase`. Esse caso de uso valida que o solicitante autenticado é o dono da solicitação, que a solicitação ainda não expirou e que sua meta ainda não foi atingida; em seguida, delega ao `FindEligibleDonorsUseCase` (por meio do serviço de domínio `DonorMatchingService`) a identificação dos doadores elegíveis — considerando compatibilidade sanguínea e elegibilidade temporal — para, então, notificar cada doador elegível com conta ativa através do serviço de notificação descrito na Seção 5.5.3. Esse fluxo materializa, na arquitetura da API, o comportamento observado na literatura (Capítulo 3) de notificar diretamente candidatos compatíveis em vez de aguardar passivamente novas doações.

5.6.3 Materialização do cumprimento de metas (`fulfilledBloodBags`)

Não existe um vínculo direto entre uma doação e uma solicitação específica: cada doação concluída em um hemocentro entra em um agrupamento (*pool*) de doações daquela organização e é distribuída entre as solicitações ativas do mesmo hemocentro pelo `DonationRequestFulfillmentService` seguindo uma regra de alocação *First In, First Out* (FIFO) por hemocentro: as solicitações são ordenadas da mais antiga para a mais nova e, para cada doação (também ordenada cronologicamente), o serviço percorre as solicitações compatíveis — mesmo tipo sanguíneo, dentro da janela entre a data de solicitação e a data-limite, e com meta ainda não atingida — alocando a doação à primeira solicitação elegível. A verificação de compatibilidade entre uma doação e uma solicitação específica é encapsulada no método `DonationRequest.acceptsDonation(...)`, preservando a regra de negócio dentro do agregado.

Diferentemente da estratégia inicialmente cogitada de recalcular essa alocação a cada leitura, optou-se por materializar o resultado no momento da escrita: o campo `fulfilledBloodBags` é persistido na própria solicitação e recalculado apenas quando o conjunto de doações concluídas de um hemocentro é alterado. Esse recálculo é disparado pelo `DonationRequestFulfillmentRefresher` a partir de dois casos de uso — `CreateCompletedDonationUseCase` e `CompletePendingDonationUseCase` —, que, após persistirem a doação, carregam as solicitações ativas e as doações concluídas da organização afetada, executam o método `synchronize(...)` do serviço de *fulfillment* e persistem cada solicitação com o novo valor calculado.

Essa abordagem caracteriza um contador materializado (*materialized/denormalized counter*): as leituras de recomendação e de listagem de solicitações por participante apenas consultam o campo já calculado, enquanto o custo do algoritmo de alocação é pago somente nas escritas que efetivamente alteram o cenário de doações concluídas — criação de solicitação, cancelamento e alterações de meta ou prazo não disparam o recálculo, permanecendo consistentes até a próxima doação concluída daquele hemocentro. Trata-se de uma decisão de desempenho deliberada: como as leituras (recomendações e listagens) tendem a ser bem mais frequentes do que

a conclusão de doações, deslocar o custo computacional do caminho de leitura para o caminho de escrita reduz a carga da API sem comprometer a corretude do algoritmo FIFO, que continua sendo recalculado por completo — e não incrementado ingenuamente — a cada sincronização.

6 Avaliação da proposta de solução

Este capítulo descreve o plano de avaliação da API desenvolvida, alinhado ao objetivo específico O5 (avaliar a manutenibilidade da solução) e ao objetivo O4 (validar a proposta por meio de estudo de caso e implementação). Como a segunda etapa do trabalho ainda está em andamento, as seções a seguir apresentam o que será avaliado e como, e não os resultados finais, que serão consolidados na versão definitiva desta monografia.

6.1 Plano de avaliação

A avaliação da solução será conduzida em duas frentes complementares. A primeira, de caráter funcional, apoia-se em testes automatizados: testes unitários para as regras de negócio isoladas no domínio (por exemplo, elegibilidade do doador, compatibilidade sanguínea e a alocação FIFO do `DonationRequestFulfillmentService`) e testes de integração para os fluxos que atravessam as camadas de aplicação, persistência e segurança (como a criação de uma solicitação seguida de doações concluídas e a consequente atualização de `fulfilledBloodBags`). A segunda frente, de caráter estrutural, avalia a manutenibilidade da API à luz dos princípios SOLID e do DDD apresentados no Capítulo 2, verificando se a separação em camadas (`domain`, `application/usecase`, `interfaces/rest`, `infra/persistence` e `infra/security`) de fato isola as regras de negócio de detalhes de infraestrutura, conforme descrito no Capítulo 5.

6.2 Resultados

Esta seção apresentará, na versão final do trabalho, a cobertura e os resultados obtidos com a suíte de testes unitários e de integração, bem como eventuais evidências qualitativas de manutenibilidade (como a facilidade de estender um novo caso de uso ou de substituir uma implementação de infraestrutura sem alterar o domínio). No momento da entrega desta versão, a implementação da API e a suíte de testes correspondente ainda estão em desenvolvimento, de modo que os resultados quantitativos não são apresentados aqui.

6.3 Sugestões de melhorias

A partir dos resultados da avaliação, esta seção reunirá melhorias identificadas ao longo da validação — por exemplo, ajustes no algoritmo de alocação de doações, na cobertura de testes de casos de borda (solicitações expiradas, doações concorrentes) ou na experiência de consumo da API evidenciada pela documentação OpenAPI. Como a avaliação ainda não foi concluída, este trabalho não antecipa melhorias específicas nesta versão.

6.4 Considerações sobre a avaliação

Ao final da avaliação, esta seção sintetizará em que medida a API construída atendeu aos objetivos O3, O4 e O5, discutindo a eficácia da estratégia de recomendação e de cumprimento de metas descrita no Capítulo 5 e o grau de manutenibilidade alcançado pela arquitetura em camadas orientada a DDD. Essa síntese será redigida somente após a execução do plano de avaliação descrito neste capítulo.

7 Considerações finais

Este capítulo retomará, na versão final do trabalho, a problemática apresentada na Introdução à luz dos resultados obtidos com o desenvolvimento e a avaliação da API. Como a avaliação descrita no Capítulo 6 ainda está em andamento, as seções a seguir estruturam o raciocínio que orientará essas considerações, sem antecipar conclusões sobre resultados ainda não obtidos.

7.1 Alcance dos objetivos

Os objetivos específicos O1 (mapeamento do processo de doação com DDD) e O2 (identificação de sistemas similares) já foram integralmente endereçados, respectivamente, pela modelagem de domínio apresentada no Capítulo 5 e pelo mapeamento sistemático da literatura conduzido no Capítulo 3. O objetivo O3 (mecanismos de recomendação e notificação) foi implementado e descrito na Seção 5.6, restando avaliar seu comportamento em cenários mais amplos. Os objetivos O4 (validação por estudo de caso) e O5 (avaliação de manutenibilidade) dependem diretamente do plano de avaliação do Capítulo 6; o alcance desses dois últimos objetivos será formalmente discutido nesta seção após a conclusão dos testes e da análise de manutenibilidade.

7.2 Principais contribuições

Do ponto de vista acadêmico, este trabalho contribui com um mapeamento sistemático da literatura sobre sistemas de captação de doadores de sangue (Capítulo 3) e com um estudo de caso detalhado de aplicação de DDD e princípios SOLID a um domínio de saúde pública com regras de elegibilidade e compatibilidade sanguínea. Do ponto de vista tecnológico, a principal contribuição é a própria API RESTful, incluindo a estratégia de recomendação de solicitações e a materialização do cumprimento de metas por meio de alocação FIFO (Seção 5.6), que pode servir de referência para soluções semelhantes. As contribuições específicas de cada frente serão detalhadas nesta seção conforme os resultados do Capítulo 6 forem consolidados.

7.3 Trabalhos futuros

Como direcionamentos para evoluções futuras, destacam-se: (i) a extensão do recálculo de `fulfilledBloodBags` para eventos além da conclusão de doações (criação, cancelamento e alteração de solicitações), reduzindo a janela de inconsistência descrita na Seção 5.6; (ii) a evolução do algoritmo de recomendação para considerar critérios adicionais, como o histórico

de resposta do doador a notificações anteriores; (iii) a extração de módulos mais explicitamente isolados para os contextos delimitados hoje tratados apenas por convenção de nomenclatura (Seção 5.4.1); e (iv) a ampliação da suíte de testes automatizados e da avaliação de manutenibilidade para outras versões da API, à medida que novas funcionalidades forem incorporadas.

Referências

- ALESSA, T. Evaluation of the wateen app in the blood-donation process in saudi arabia. *Journal of Blood Medicine*, v. 13, p. 181–190, 2022.
- BLOCH, J. *Effective Java*. 3. ed. Boston: Addison-Wesley, 2018.
- BOOCH, G. *Object-oriented analysis and design with applications*. 3. ed. Boston: Addison-Wesley, 2007.
- Ceará. Secretaria da Saúde. *Homens são os que mais doam sangue no Ceará entre os 2% da população doadora*. Fortaleza: [s.n.], 2021. Online.
- Ceará. Secretaria da Saúde. *Hemoce registra número recorde de doações em 2024, o maior nos últimos 30 anos*. Fortaleza: [s.n.], 2025. Online.
- Centro de Hematologia e Hemoterapia do Ceará — HEMOCE. *Rede de atendimento*. Fortaleza: [s.n.], 2022. Online.
- CHACON, S.; STRAUB, B. *Pro Git*. [S.l.]: Apress, 2014.
- DEITEL, P.; DEITEL, H. *Java: How to Program*. [S.l.]: Pearson, 2017.
- Elsevier. *Scopus*. [S. l.]: Elsevier, 2024. Online.
- Elsevier. *Scopus content coverage*. 2025. Online. Mais de 2,4 bilhões de referências citadas desde 1970.
- EVANS, E. *Domain-Driven Design: Tackling Complexity in the Heart of Software*. [S.l.]: Addison-Wesley, 2003.
- FRANDSEN, F. *Health sciences literature review made easy: The PICO strategy*. Burlington: Jones & Bartlett Learning, 2020.
- GAMMA, E. et al. *Design patterns: elements of reusable object-oriented software*. Reading: Addison-Wesley, 1994.
- GIL, A. C. *Métodos e técnicas de pesquisa social*. 6. ed. São Paulo: Atlas, 2008.
- Hemorio. *Doação de sangue no Brasil: desafios e perspectivas*. Rio de Janeiro, 2023.
- LI, L. et al. Mobile applications for encouraging blood donation: A systematic review and case study. 2023.
- MARCONI, M. d. A.; LAKATOS, E. M. *Fundamentos de metodologia científica*. 5. ed. São Paulo: Atlas, 2003.
- MARTIN, R. C. *Design principles and design patterns*. [S.l.], 2000.
- MARTIN, R. C. *Agile software development: principles, patterns, and practices*. Upper Saddle River: Prentice Hall, 2003.

- MARTIN, R. C. *Clean code: a handbook of agile software craftsmanship*. Upper Saddle River: Prentice Hall, 2008.
- MARTIN, R. C. *Clean architecture: a craftsman's guide to software structure and design*. Boston: Prentice Hall, 2017.
- MINAYO, M. C. d. S. *Pesquisa social: teoria, método e criatividade*. 19. ed. Petrópolis: Vozes, 2001.
- Organização Mundial da Saúde. *Blood safety and availability*. Geneva, 2020.
- Organização Pan-Americana da Saúde. *Supply of blood for transfusion in Latin American and Caribbean countries 2019-2020*. Washington, D.C., 2020.
- OU-YANG, J. et al. Effective methods for reactivating inactive blood donors: A stratified randomised controlled study. 2020.
- PETERSEN, K. et al. Systematic mapping studies in software engineering. In: *12th International Conference on Evaluation and Assessment in Software Engineering (EASE)*. [S.l.: s.n.], 2008. p. 1–10.
- PRESSMAN, R. S.; MAXIM, B. R. *Engenharia de Software: uma abordagem profissional*. 8. ed. Porto Alegre: AMGH, 2016.
- PRODANOV, C. C.; FREITAS, E. C. d. *Metodologia do trabalho científico*. 2. ed. Novo Hamburgo: Feevale, 2013.
- SILVA, J. R. D. et al. Blood donation support application: Contributions from experts on the tool's functionality. *Ciência & Saúde Coletiva*, v. 26, n. 2, p. 493–503, 2021.
- SOMMERVILLE, I. *Engenharia de software*. 9. ed. São Paulo: Pearson Prentice Hall, 2011.
- VERNON, V. *Implementing Domain-Driven Design*. [S.l.]: Addison-Wesley, 2013.
- VERNON, V. *Domain-driven design distilled*. Boston: Addison-Wesley, 2016.
- WALLS, C. *Spring in Action*. [S.l.]: Manning, 2019.